Invertible Normalizing Flows for Generative Image Modeling: Executable Methods and Results Log

Matt Tivnan*

September 1, 2026

Abstract

This document is the executable methods-and-results log for the invertible normalizing flow project. Each numbered step of the pipeline is one `step*.py` script paired with one YAML config in `configs/` and one output folder under `outputs/`; the tables and figures below are included directly from those output folders, so the compiled PDF is an exact record of what was run and what it produced. Step 1 (this version) builds PyTorch datasets and dataloaders with deterministic train/val/test splits for the standard generative-image-modeling benchmarks: MNIST, FashionMNIST, CIFAR-10/100, SVHN, the MedMNIST 2D collection, CelebA, Imagenette, ImageNet, AFHQ, FFHQ, CelebA-HQ, and LSUN bedroom.

1 Execution environment

- Host: `axis03`; container: `recon-dev` (image `localhost:5000/recon:acr-20260819-v1`; PyTorch 2.11.0+cu128, torchvision 0.26.0, CUDA 12.4, 2 GPUs).
- Repository mounted at `/dev_ws/invertible_normalizing_flow_20260831` inside the container (shared workspace `/home/staticct/recon-dev/workspace` on the host).
- Datasets live on the axis03 data drive at `/data/image_benchmarks`, mounted into the container at the same path. This is a shared local repository of public benchmarks, reused across projects; nothing dataset-sized is stored in git.
- All steps are run as `docker exec recon-dev python /dev_ws/invertible_normalizing_flow_20260831/s`
- Exact package versions, the resolved config, and the command line for the run shown in this PDF are recorded in `outputs/step1_datasets/data/provenance.json`.

2 Step 1: Datasets and dataloaders

2.1 Methods

- Code: `step1_datasets.py` (repository root). Config: `configs/step1_datasets.yml`. Outputs: `outputs/step1_datasets/{data,figures}/`.
- Command executed for this document: `python step1_datasets.py` (runs every dataset enabled in the config).

*Repository: `github.com/tivnanmatt/invertible_normalizing_flow_20260831`

- The script exposes a library API used by later steps: `build_dataset(name, cfg)` returns a bundle of three `torch.utils.data.Dataset` splits plus metadata, and `get_dataloaders(bundle, cfg)` wraps them in `DataLoader`s (batch size, workers, pinning from the config).
- Auto-download datasets: MNIST, FashionMNIST, CIFAR-10, CIFAR-100, SVHN (torchvision); the MedMNIST 2D collection (`medmnist` package, official train/val/test); Imagenette (fast.ai 10-class ImageNet subset, used as a fast stand-in for ImageNet); AFHQ (StarGAN-v2 release, fetched from the official Dropbox link); CelebA (torchvision Google-Drive download, quota permitting).
- Manual-download datasets (access terms or hosting): ImageNet (ILSVRC2012), FFHQ, CelebA-HQ, LSUN bedroom. When absent, the script records them as *unavailable* together with exact acquisition instructions (Section 2.4) instead of failing.
- Train/val/test policy (deterministic, seeded from the config): official splits are used whenever they exist (MedMNIST, CelebA); datasets with only train/test get a validation set carved from train (10%); datasets whose public split is train/val only (Imagenette, ImageNet, AFHQ) use the official val as *test* and carve val from train; single-pool datasets (FFHQ, CelebA-HQ) are split 80/10/10.
- Transforms: `ToTensor` to $[0, 1]$ floats only; variable-size natural-image datasets additionally get `Resize`+`CenterCrop` to the per-dataset `image_size` in the config. No normalization or dequantization is baked in here – for flow training those belong to the model step so the data step stays lossless.
- Per dataset the run itself is the test: all three splits are instantiated, real batches are pulled through the dataloaders, per-channel mean/std are computed over up to 20 train batches, and three figures are written (sample grid per split, pixel-intensity histogram, class distribution where integer labels exist).
- Machine-readable outputs: one JSON per dataset, `summary.json`, `summary.csv`, `provenance.json`, and the LaTeX fragments (`summary_table.tex`, `figures_step1.tex`, `unavailable_step1.tex`) that are included verbatim below.

2.2 Configuration (verbatim)

```
# Config for step1_datasets.py -- datasets & dataloaders.
#
# data_root is the shared benchmark-dataset repository on the axis03 data
# drive, mounted into the recon-dev container at the same path
# (/data/image_benchmarks). Datasets download/live there once and are reused
# across projects; nothing dataset-sized lands in the git repo.

seed: 20260831
data_root: /data/image_benchmarks
output_root: outputs/step1_datasets

loader:
  batch_size: 64
  num_workers: 4
  pin_memory: true
```

```yaml
split:
  # carved from the official train pool when no official val split exists
  val_fraction: 0.10
  # only used for single-pool datasets with no official splits (ffhq, celeba_hq)
  test_fraction: 0.10

figures:
  samples_per_split: 8     # images per row in the sample-grid figure
  stats_max_batches: 20    # batches used for per-channel mean/std

datasets:
  # ---- small standard benchmarks (auto-download) ----
  mnist:         {enabled: true}
  fashion_mnist: {enabled: true}
  cifar10:       {enabled: true}
  cifar100:      {enabled: true}
  svhn:          {enabled: true}

  # ---- MedMNIST 2D collection (auto-download) ----
  # All 12 2D members are supported; the run set below spans the modalities
  # (pathology, blood smear, dermoscopy, X-ray, ultrasound, fundus, abdominal
  # CT). octmnist/tissuemnist/chestmnist are large; enable by adding flags.
  medmnist:
    enabled: true
    size: 28
    flags:
      - pathmnist
      - bloodmnist
      - dermamnist
      - pneumoniamnist
      - breastmnist
      - retinamnist
      - organamnist

  # ---- faces / natural images ----
  celeba:          # aligned CelebA; auto-download tried, gdrive quota permitting
    enabled: true
    image_size: 64
  imagenette:      # fast.ai 10-class ImageNet subset; auto-download stand-in
    enabled: true  # for full ImageNet in quick experiments
    variant: 160px
    image_size: 64
  imagenet:        # full ILSVRC2012; manual download (access terms)
    enabled: true
    image_size: 64
  afhq:            # animal faces (cat/dog/wild), StarGAN-v2 release
    enabled: true
    version: v1
    image_size: 64
    attempt_download: true
  ffhq:            # 70k faces; manual download (NVlabs)
    enabled: true
    image_size: 128
  celeba_hq:       # 30k faces; manual download
```

```
    enabled: true
    image_size: 256
  lsun_bedroom:    # LSUN lmdb; manual download
    enabled: true
    image_size: 64
```

2.3 Results: dataset summary

dataset	status	train	val	test	image	classes
mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
fashion_mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
cifar10	ok	45000	5000	10000	$3 \times 32 \times 32$	10
cifar100	ok	45000	5000	10000	$3 \times 32 \times 32$	100
svhn	ok	65931	7326	26032	$3 \times 32 \times 32$	10
medmnist_pathmnist	ok	89996	10004	7180	$3 \times 28 \times 28$	9
medmnist_bloodmnist	ok	11959	1712	3421	$3 \times 28 \times 28$	8
medmnist_dermamnist	ok	7007	1003	2005	$3 \times 28 \times 28$	7
medmnist_pneumoniamnist	ok	4708	524	624	$1 \times 28 \times 28$	2
medmnist_breastmnist	ok	546	78	156	$1 \times 28 \times 28$	2
medmnist_retinamnist	ok	1080	120	400	$3 \times 28 \times 28$	5
medmnist_organamnist	ok	34561	6491	17778	$1 \times 28 \times 28$	11
celeba	unavailable	–	–	–	–	–
imagenette	ok	8522	947	3925	$3 \times 64 \times 64$	10
imagenet	unavailable	–	–	–	–	–
afhq	ok	13167	1463	1500	$3 \times 64 \times 64$	3
ffhq	unavailable	–	–	–	–	–
celeba_hq	unavailable	–	–	–	–	–
lsun_bedroom	unavailable	–	–	–	–	–

Split sizes are reported after the split policy above; *image* is the tensor shape $C \times H \times W$ delivered by the dataloaders; *classes* is the number of integer classes ("–" for unconditional / multi-label datasets).

2.4 Results: unavailable datasets

- **celeba**: CelebA auto-download failed (Google Drive quota is a known issue: RuntimeError). Manually place img_align_celeba/ plus the list_*.txt / identity_* files under data/celeba/celeba/ (see torchvision.datasets.CelebA docs), then re-run.
- **imagenet**: ImageNet (ILSVRC2012) requires manual download (terms of access): obtain from https://image-net.org, extract to data/imagenet/train/<wnid>/*.JPEG and data/imagenet/val/<wnic then re-run.
- **ffhq**: FFHQ requires manual download: place images (e.g. thumbnails128x128 or images1024x1024 from github.com/NVlabs/ffhq-dataset) under data/ffhq/
- **celeba_hq**: CelebA-HQ requires manual download: place the 30k CelebA-HQ images (e.g. celeba_hq_256 from the progressive-GAN release) under data/celeba_hq/

- **lsun_bedroom**: LSUN bedroom requires manual download: fetch bedroom_train_lmdb / bedroom_val_lmdb with github.com/fyu/lsun download.py into data/lsun/ (also needs 'pip install lmdb').

2.5 Results: figures

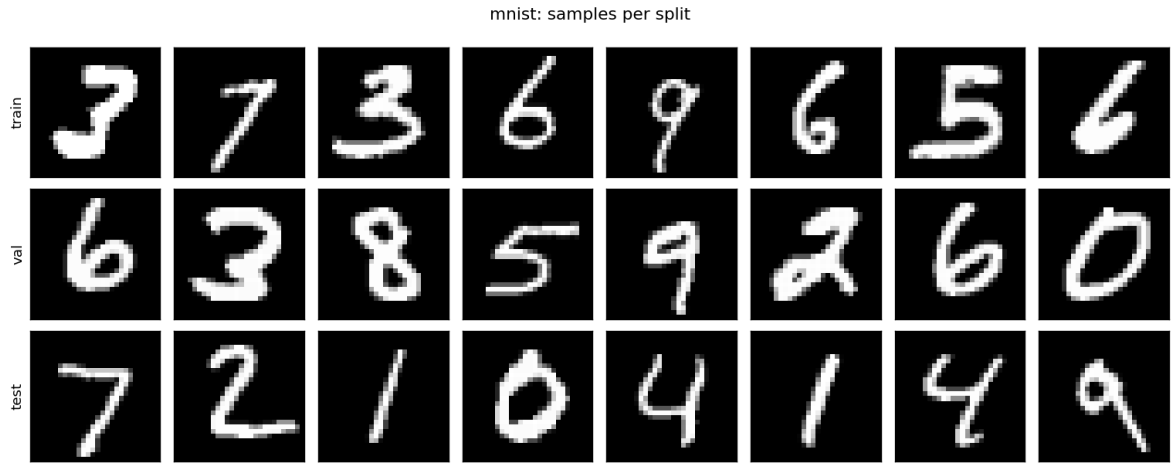

Figure 1: **mnist** (samples). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_samples.png`, produced by `step1_datasets.py`.

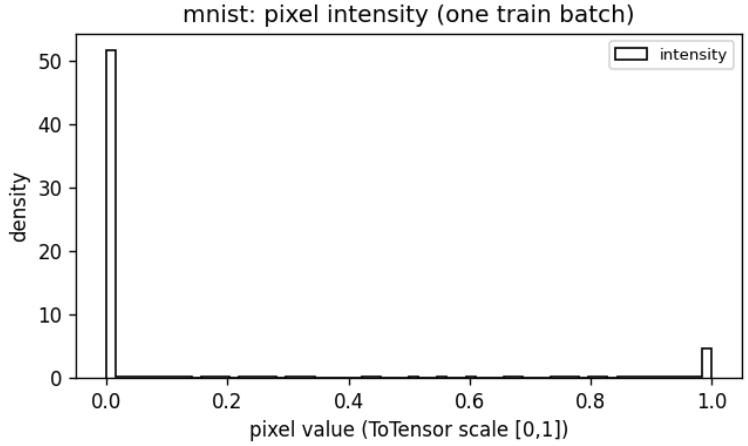

Figure 2: **mnist** (hist). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_pixel_hist.png`, produced by `step1_datasets.py`.

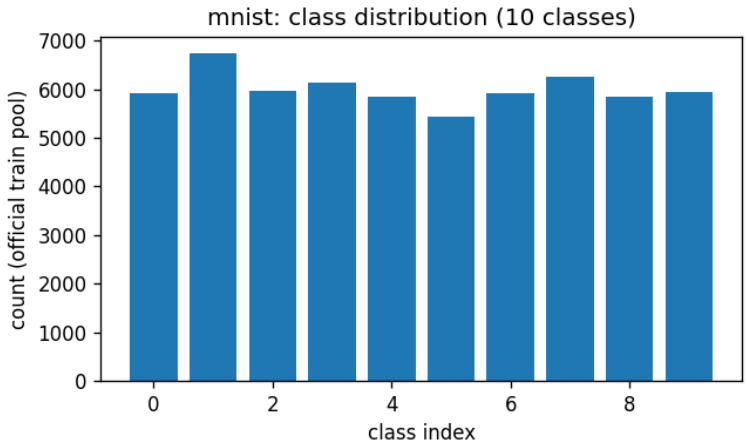

Figure 3: **mnist** (dist). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_class_dist.png`, produced by `step1_datasets.py`.

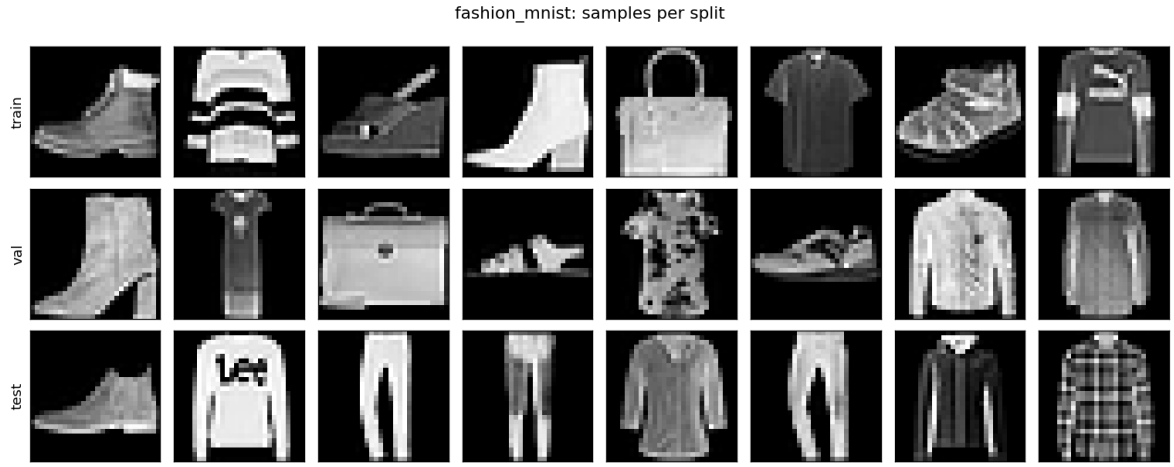

Figure 4: **fashion_mnist** (samples). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/fashion_mnist_samples.png`, produced by `step1_datasets.py`.

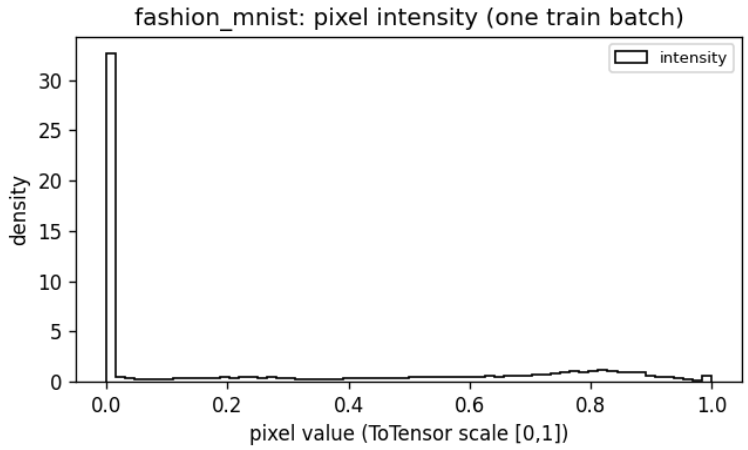

Figure 5: **fashion_mnist** (hist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/fashion_mnist_pixel_hist.png`, produced by `step1_datasets.py`.

Figure 6: **fashion_mnist** (dist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/fashion_mnist_class_dist.png`, produced by `step1_datasets.py`.

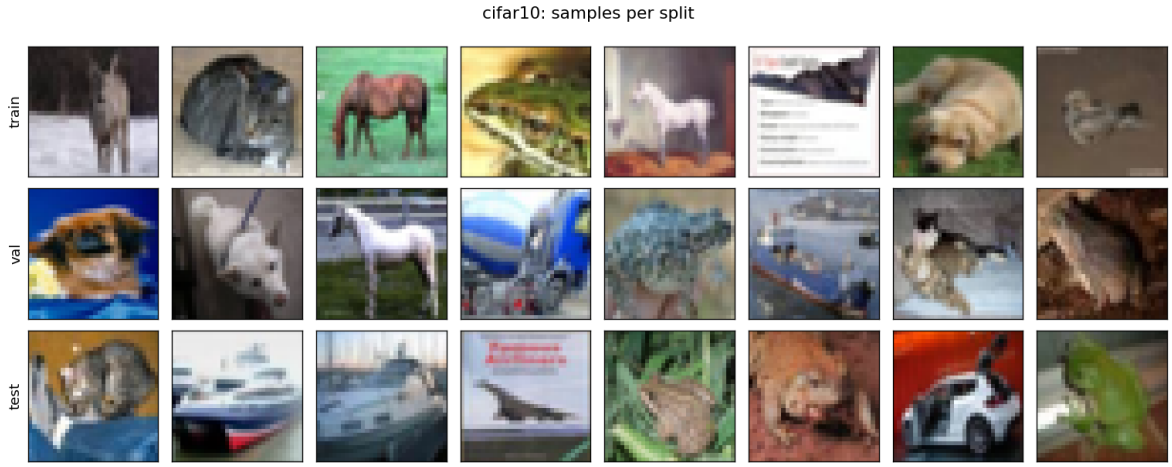

Figure 7: **cifar10** (samples). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar10_samples.png`, produced by `step1_datasets.py`.

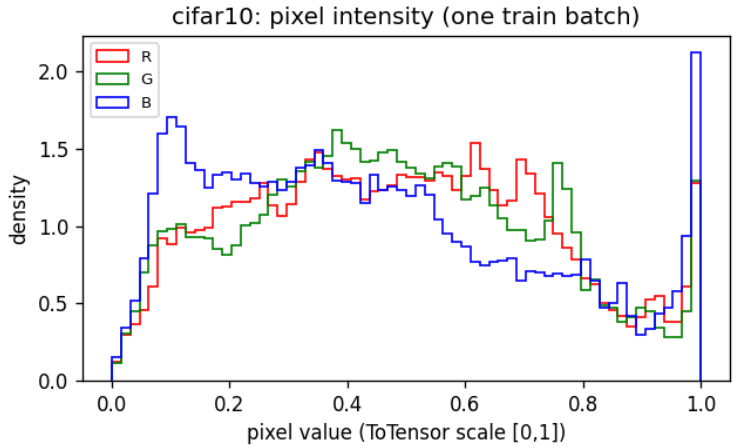

Figure 8: **cifar10** (hist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar10_pixel_hist.png`, produced by `step1_datasets.py`.

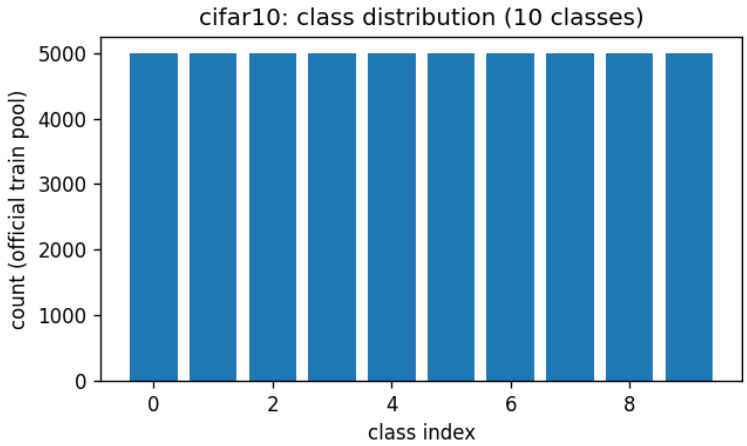

Figure 9: **cifar10** (dist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar10_class_dist.png`, produced by `step1_datasets.py`.

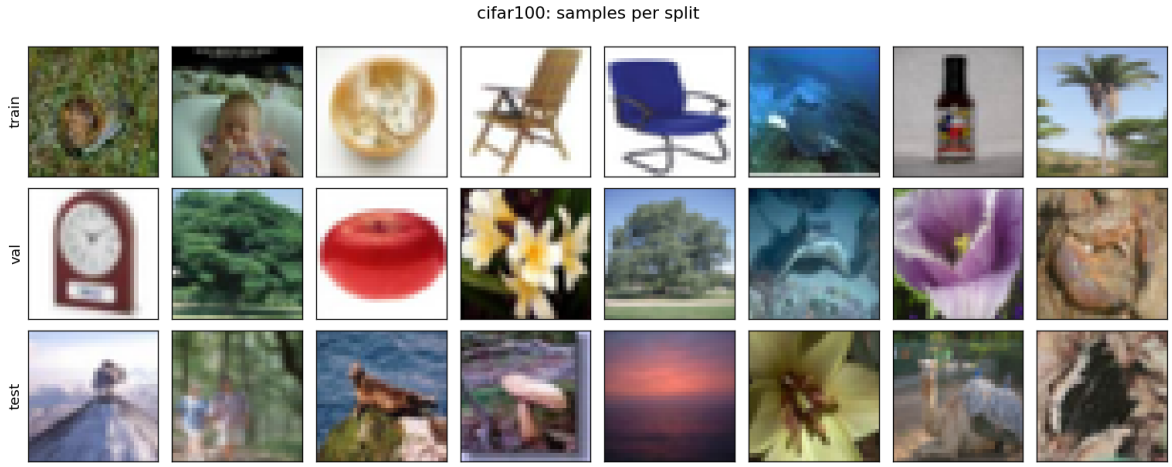

Figure 10: **cifar100** (samples). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar100_samples.png`, produced by `step1_datasets.py`.

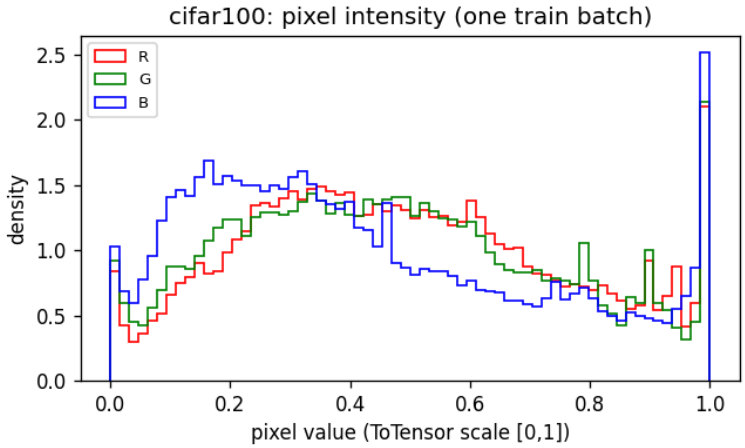

Figure 11: **cifar100** (hist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar100_pixel_hist.png`, produced by `step1_datasets.py`.

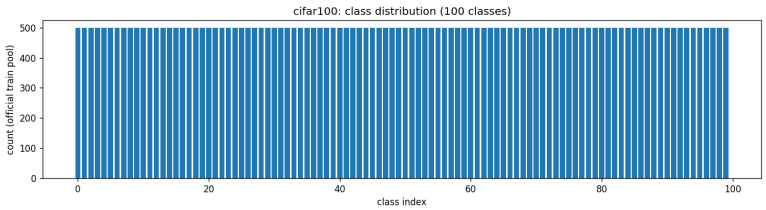

Figure 12: **cifar100** (dist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar100_class_dist.png`, produced by `step1_datasets.py`.

Figure 13: **svhn** (samples). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/svhn_samples.png`, produced by `step1_datasets.py`.

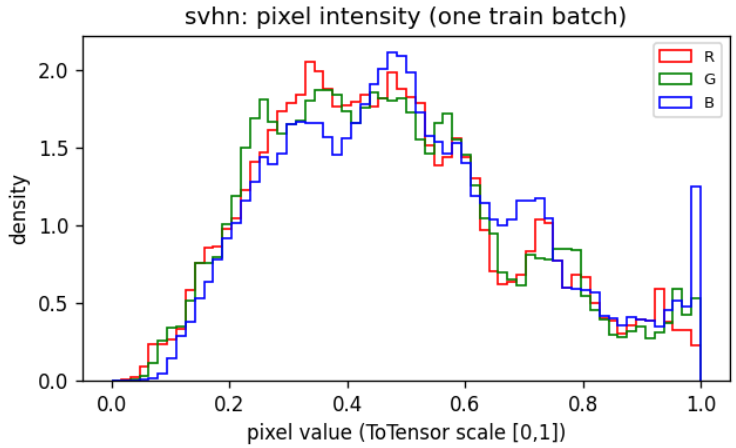

Figure 14: **svhn** (hist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/svhn_pixel_hist.png`, produced by `step1_datasets.py`.

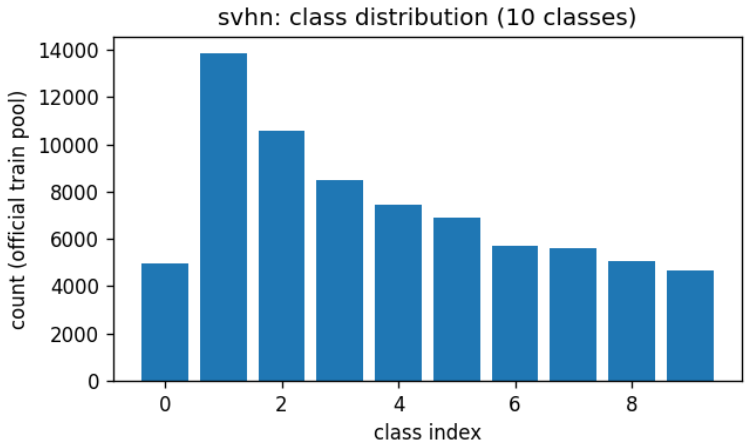

Figure 15: **svhn** (dist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/svhn_class_dist.png`, produced by `step1_datasets.py`.

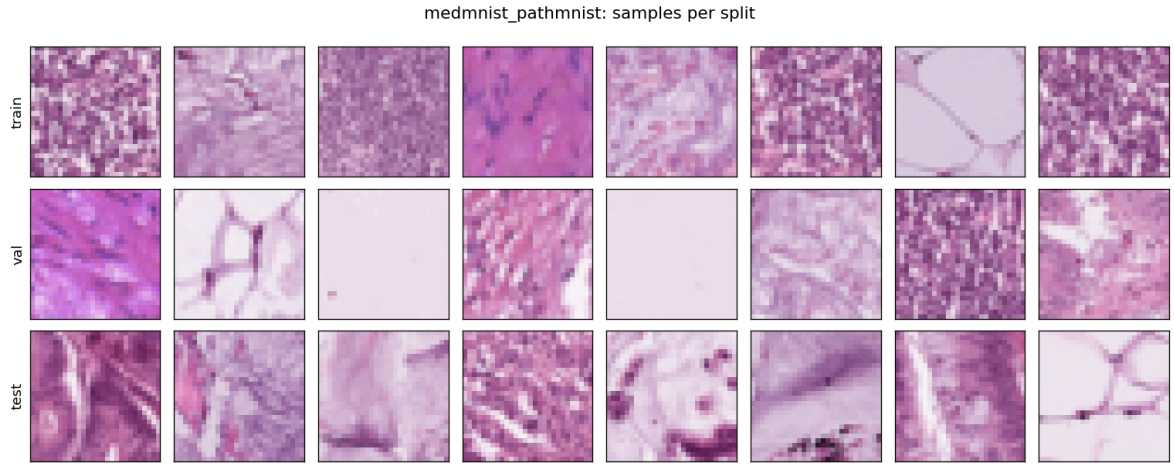

Figure 16: **medmnist_pathmnist** (samples). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pathmnist_samples.png`, produced by `step1_datasets.py`.

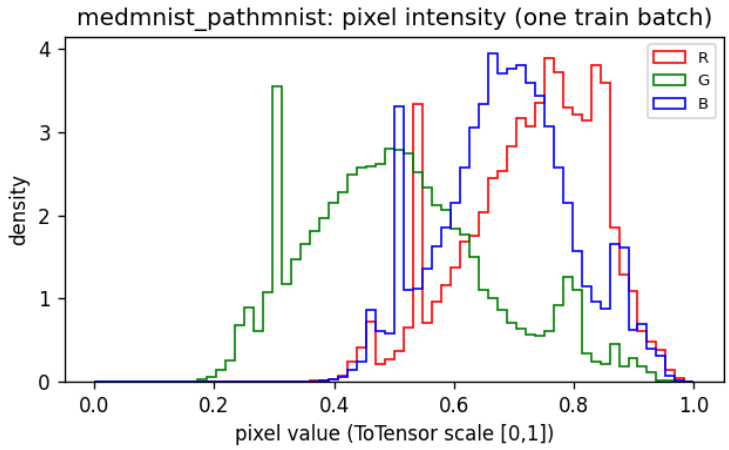

Figure 17: **medmnist_pathmnist** (hist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pathmnist_pixel_hist.png`, produced by `step1_datasets.py`.

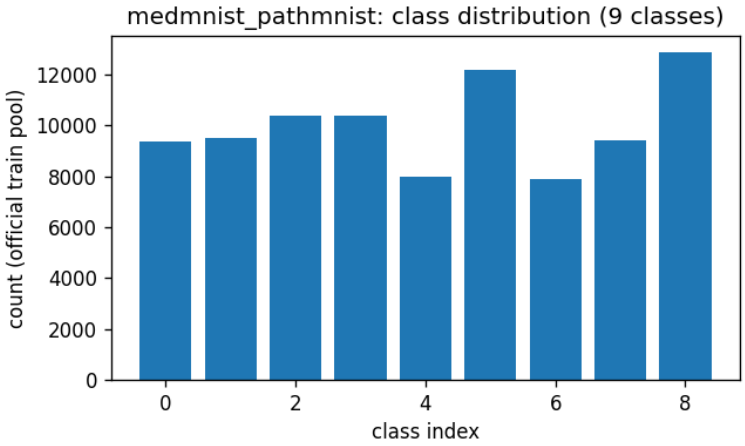

Figure 18: **medmnist_pathmnist** (dist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pathmnist_class_dist.png`, produced by `step1_datasets.py`.

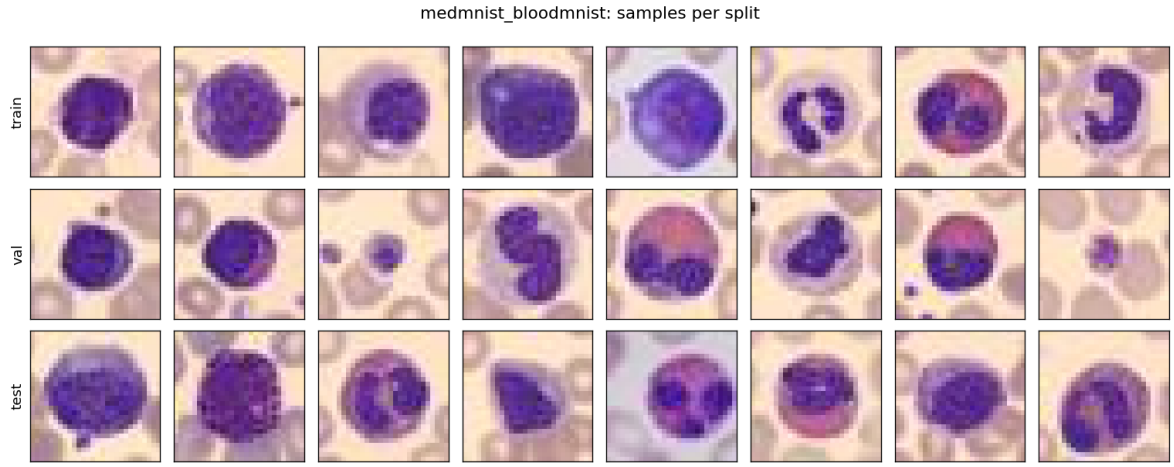

Figure 19: **medmnist_bloodmnist** (samples). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_bloodmnist_samples.png`, produced by `step1_datasets.py`.

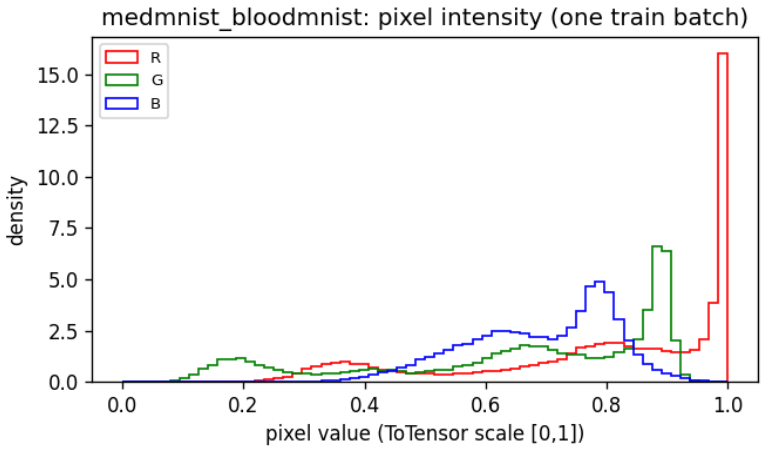

Figure 20: **medmnist_bloodmnist** (hist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_bloodmnist_pixel_hist.png`, produced by `step1_datasets.py`.

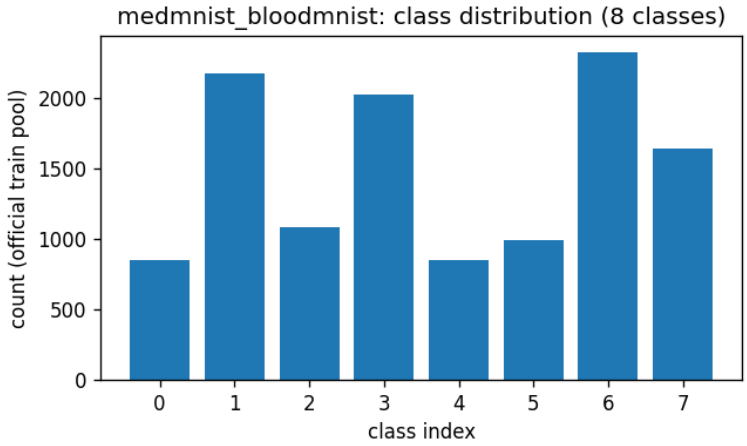

Figure 21: **medmnist_bloodmnist** (dist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_bloodmnist_class_dist.png`, produced by `step1_datasets.py`.

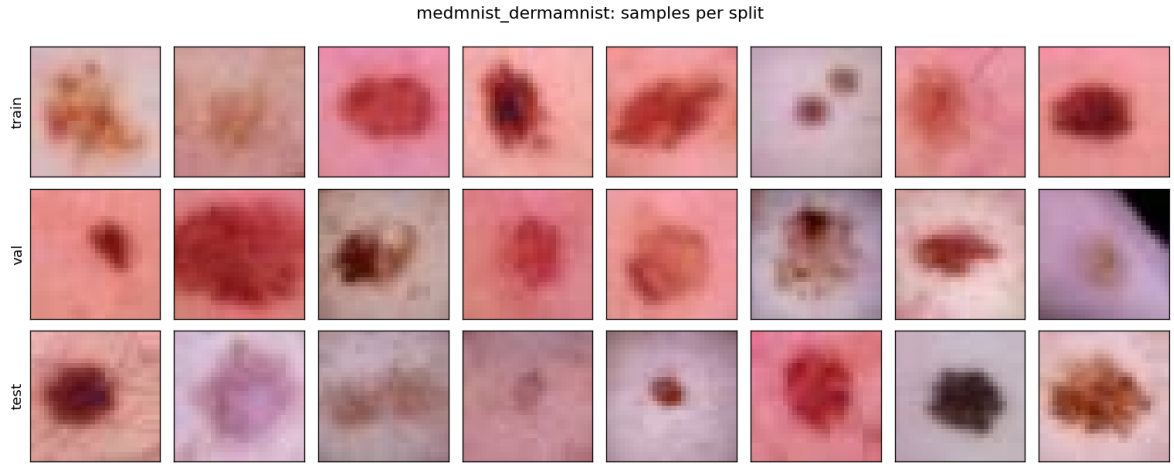

Figure 22: **medmnist_dermamnist** (samples). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_dermamnist_samples.png`, produced by `step1_datasets.py`.

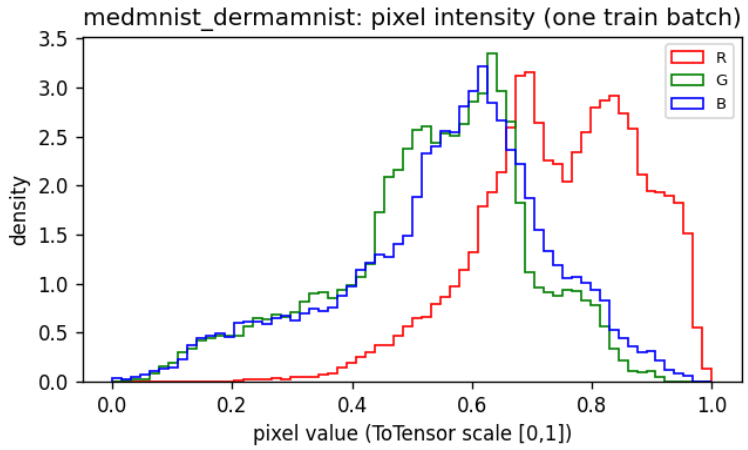

Figure 23: **medmnist_dermamnist** (hist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_dermamnist_pixel_hist.png`, produced by `step1_datasets.py`.

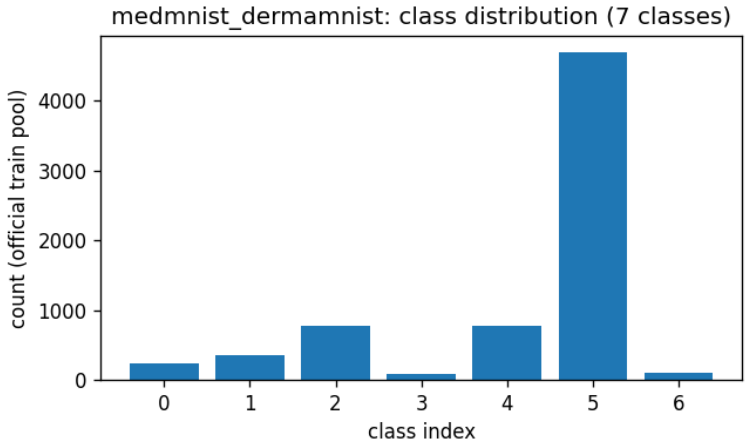

Figure 24: **medmnist_dermamnist** (dist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_dermamnist_class_dist.png`, produced by `step1_datasets.py`.

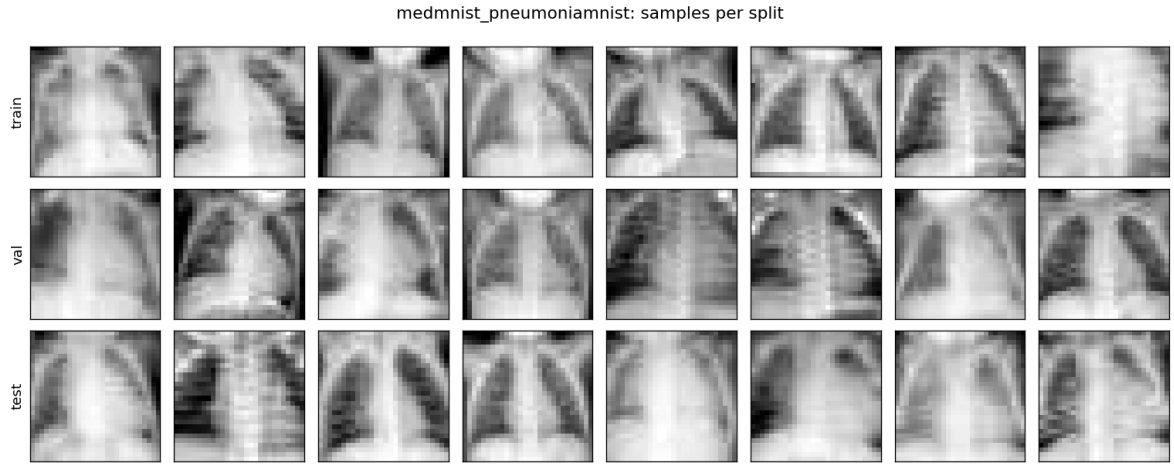

Figure 25: **medmnist_pneumoniamnist** (samples). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_samples.png`, produced by `step1_datasets.py`.

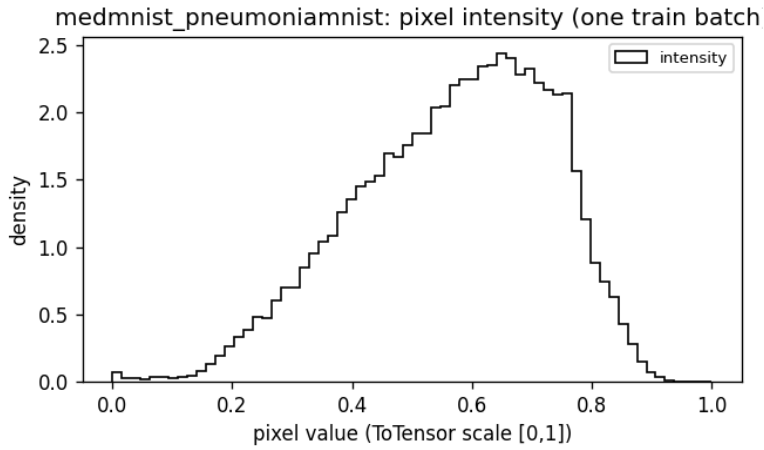

Figure 26: **medmnist_pneumoniamnist** (hist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_pixel_hist.png`, produced by `step1_datasets.py`.

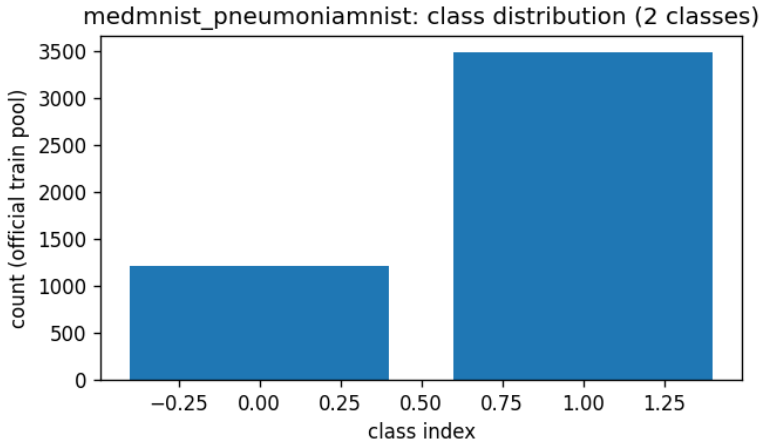

Figure 27: **medmnist_pneumoniamnist** (dist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_class_dist.png`, produced by `step1_datasets.py`.

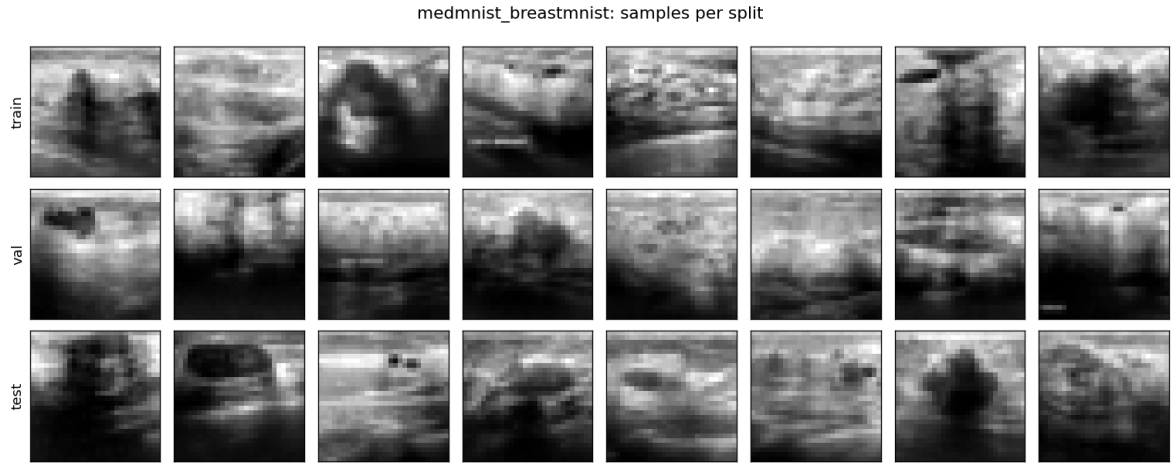

Figure 28: **medmnist_breastmnist** (samples). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_breastmnist_samples.png`, produced by `step1_datasets.py`.

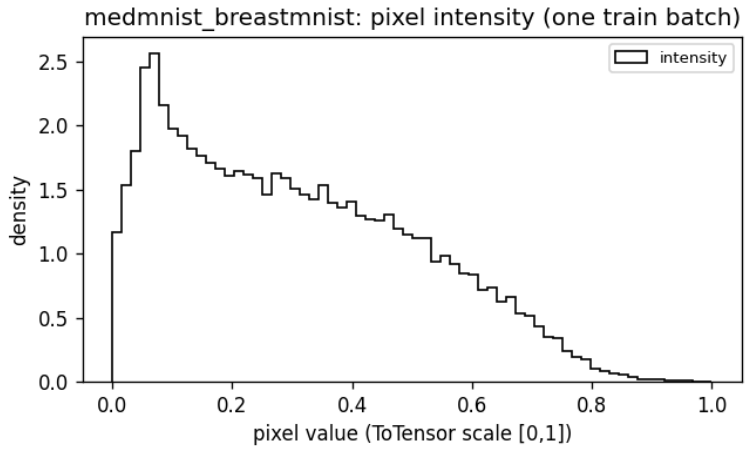

Figure 29: **medmnist_breastmnist** (hist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_breastmnist_pixel_hist.png`, produced by `step1_datasets.py`.

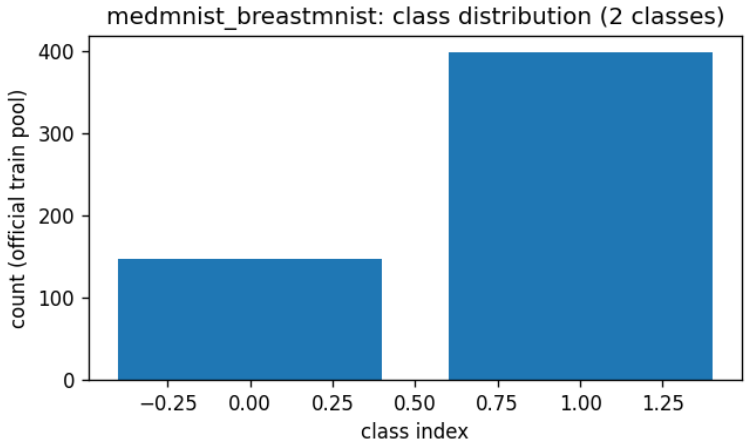

Figure 30: **medmnist_breastmnist** (dist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_breastmnist_class_dist.png`, produced by `step1_datasets.py`.

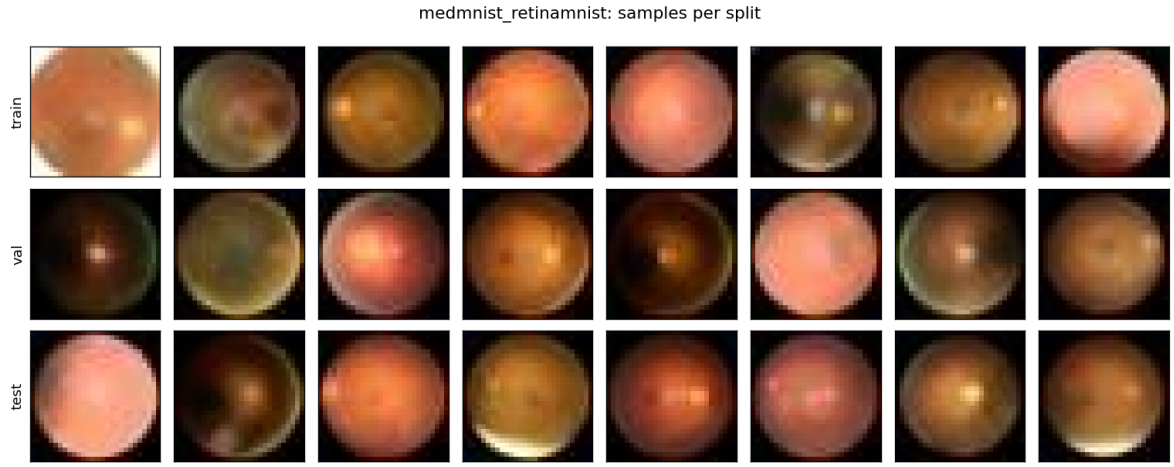

Figure 31: **medmnist_retinamnist** (samples). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_retinamnist_samples.png`, produced by `step1_datasets.py`.

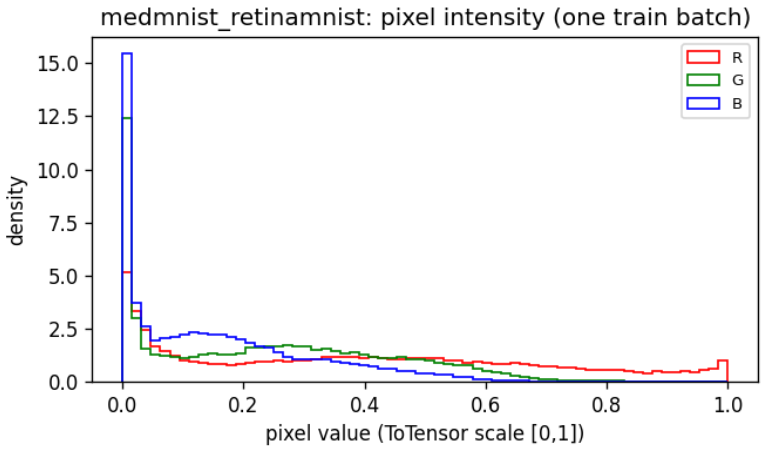

Figure 32: **medmnist_retinamnist** (hist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_retinamnist_pixel_hist.png`, produced by `step1_datasets.py`.

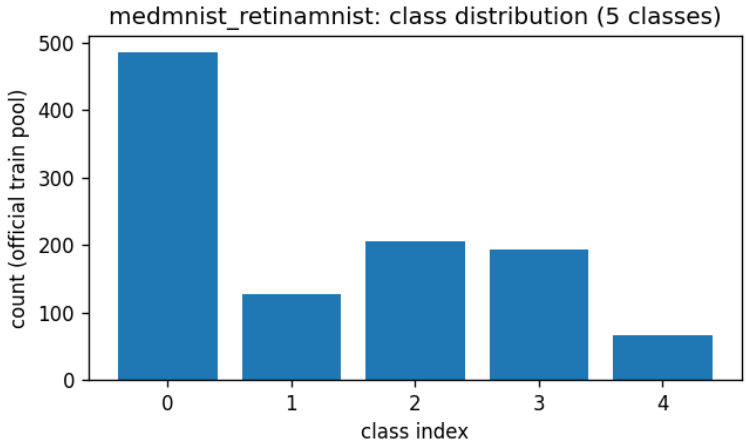

Figure 33: **medmnist_retinamnist** (dist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_retinamnist_class_dist.png`, produced by `step1_datasets.py`.

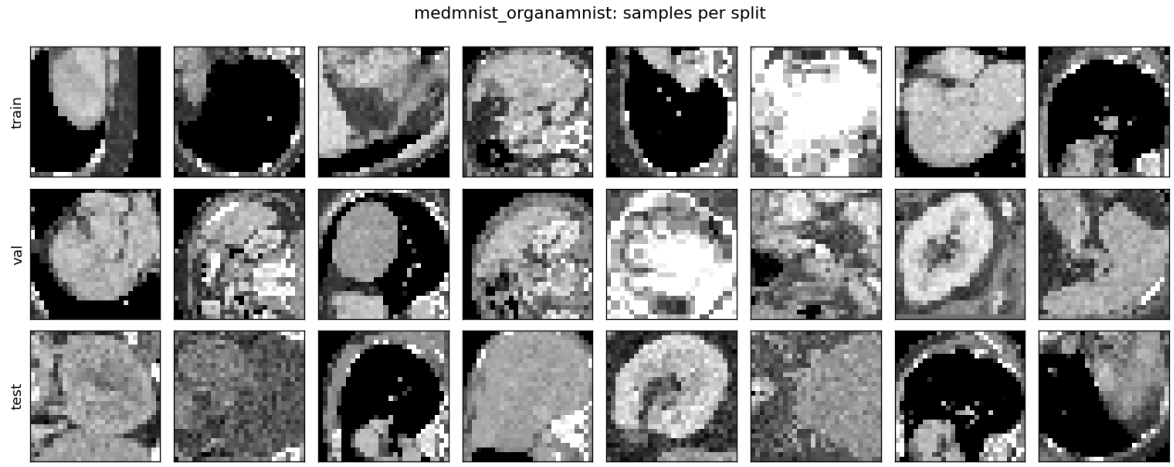

Figure 34: **medmnist_organamnist** (samples). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_organamnist_samples.png`, produced by `step1_datasets.py`.

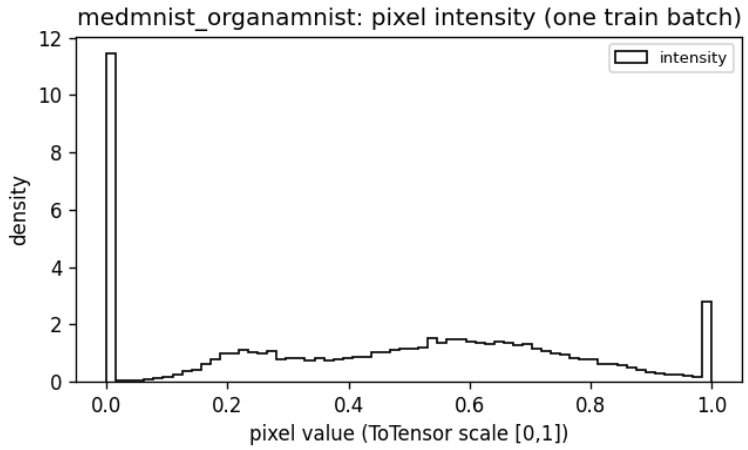

Figure 35: **medmnist_organamnist** (hist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_organamnist_pixel_hist.png`, produced by `step1_datasets.py`.

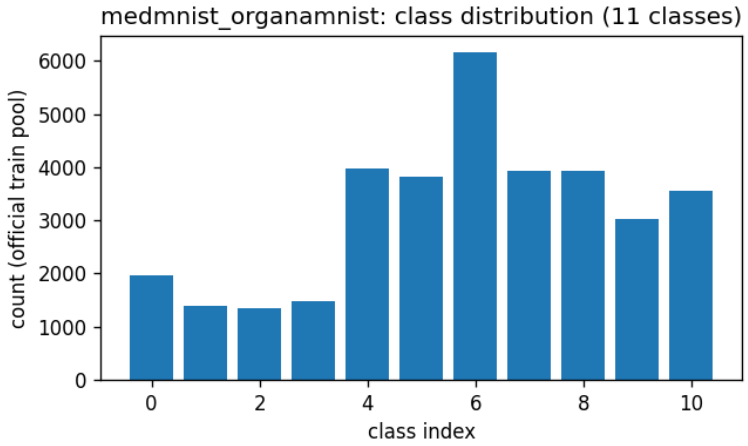

Figure 36: **medmnist_organamnist** (dist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_organamnist_class_dist.png`, produced by `step1_datasets.py`.

Figure 37: **imagenette** (samples). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_samples.png`, produced by `step1_datasets.py`.

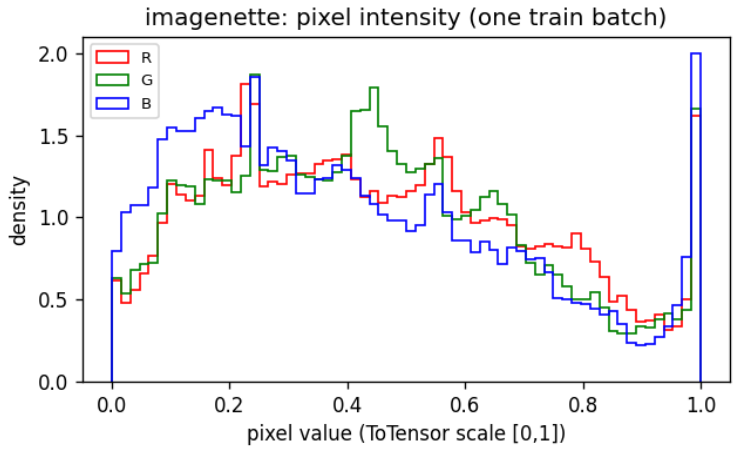

Figure 38: **imagenette** (hist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_pixel_hist.png`, produced by `step1_datasets.py`.

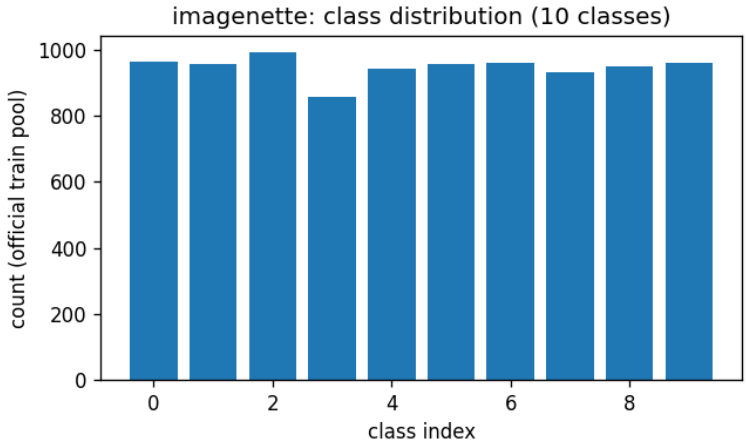

Figure 39: **imagenette** (dist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_class_dist.png`, produced by `step1_datasets.py`.

Figure 40: **afhq** (samples). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_samples.png`, produced by `step1_datasets.py`.

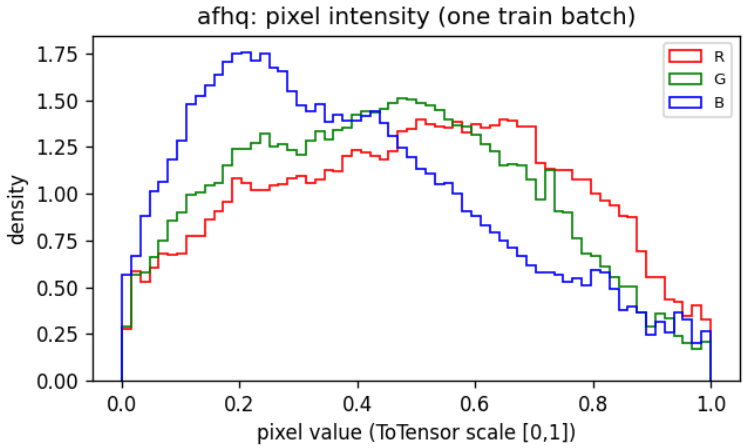

Figure 41: **afhq** (hist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_pixel_hist.png`, produced by `step1_datasets.py`.

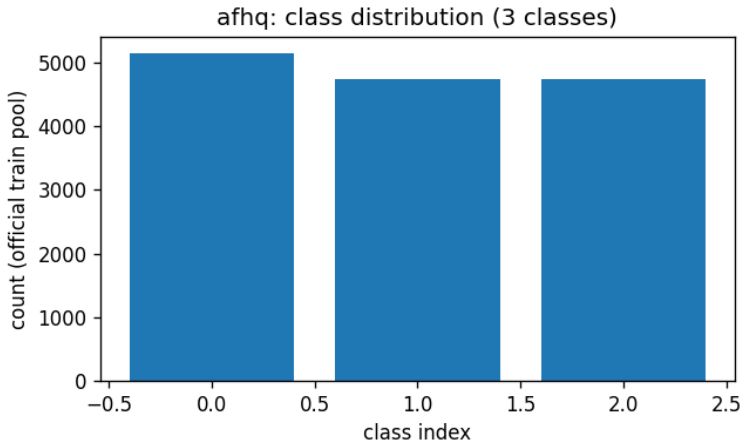

Figure 42: **afhq** (dist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_class_dist.png`, produced by `step1_datasets.py`.

2.6 Analysis

- Coverage: the available benchmarks span 28×28 grayscale (MNIST, several MedMNIST members) through 64×64 RGB natural images and faces (CIFAR, Imagenette, AFHQ, CelebA), which is the standard ladder for likelihood-based generative models: density results in bits/dim on MNIST/CIFAR-10, then perceptual-quality results on faces/animals at higher resolution.

- The pixel-intensity histograms show the strongly peaked, saturated distributions typical of 8-bit images (large masses at 0 and 1, e.g. MNIST background, CelebA skin tones). For normalizing flows this directly motivates uniform dequantization and a logit transform before the first coupling layer – to be implemented and ablated in the model step, not in the data step.

- Class-distribution figures document which datasets are usable for class-conditional modeling and how imbalanced they are; several MedMNIST members are markedly imbalanced, which matters for conditional likelihood evaluation.

- Per-channel means/stds recorded in each dataset's JSON give the constants for any input normalization later steps choose to apply.

- Datasets marked unavailable (typically ImageNet, FFHQ, CelebA-HQ, LSUN, and CelebA when the Google Drive quota blocks download) are access-restricted or manually hosted; the pipeline documents the exact acquisition path and picks them up automatically once placed under `/data/image_benchmarks`.

3 Step 2: TarFlow – Transformer Autoregressive Flow

3.1 Methods

- Model: TarFlow from *Normalizing Flows are Capable Generative Models* (Zhai et al., Apple, arXiv:2412.06329). A TarFlow is a stack of T causal-ViT "metablocks", each an autoregressive NVP flow over image patches; the patch order is flipped between consecutive blocks. Training maximizes exact likelihood; sampling inverts the blocks patch-by-patch with KV caching.

- Code: the *official* implementation (`github.com/apple/ml-tarflow`, Apple sample-code license) is used unmodified: `transformer_flow.py` is imported directly from a clone at `/dev_ws/ml-tarflow` (commit hash in `outputs/step2_tarflow/data/provenance.json`). Our driver `step2_tarflow.py` reimplements the surrounding training loop faithfully to the official `train.py` / `train_local.ipynb` / `evaluate_bpd.py`: AdamW ($\beta = (0.9, 0.95)$, wd 10^{-4}), cosine LR with one warmup epoch, inputs scaled to $[-1, 1]$, gaussian-noise runs in bf16 autocast, uniform-dequantization (likelihood) runs in fp32, and the official bits/dim formula (Gaussian prior term with the $\log 128$ scaling correction, generalized from 3 channels to C).

- Data: datasets and deterministic splits come from step 1 (`step1_datasets.build_dataset`); an adapter rescales to $[-1, 1]$ and applies the official random horizontal flip where configured.

- Config: `configs/step2_tarflow.yml`. Outputs: `outputs/step2_tarflow/{data,figures}/`. Commands executed: `python step2_tarflow.py --runs mnist_toy` (GPU 0) and `python step2_tarflow.py --runs cifar10_uniform` (GPU 1), then `python step2_tarflow.py --collect`.

- Run **mnist_toy**: exact replication of Apple's released MNIST toy experiment (`train_local.ipynb` defaults): class-conditional TarFlow [4-128-4-4] (patch-channels-blocks-layers), gaussian noise $\sigma = 0.1$, batch 256, lr 2×10^{-3}, 100 epochs. Deliverables: conditional sample grid (one row per digit) and the notebook's Bayes-rule classifier evaluation $\arg\min_y \frac{1}{2}\mathbb{E}[z_y^2] - \log|\det|_y$ on the step-1 test split.
- Run **cifar10_uniform**: unconditional CIFAR-10 likelihood run in the paper's likelihood configuration style: uniform dequantization, fp32, patch size 2 (as in their ImageNet 64 2.99-bpd config), TarFlow [2-256-8-4] ($\sim$26M params), batch 128, lr 2×10^{-4}, 60 epochs on one RTX 4090. Validation bpd is tracked every 5 epochs; final test bpd uses the official formula on the step-1 test split.
- Scale caveat (stated up front): the paper's quantitative likelihood benchmark is *unconditional ImageNet 64×64*, where TarFlow [2-768-8-8] ($\sim$470M params, 8 × A100, 200 epochs) reaches **2.99 bpd**; the paper reports no MNIST or CIFAR-10 numbers. ImageNet is not yet staged (step 1), so this step validates the framework end-to-end at small scale and positions results against the classic CIFAR-10 flow literature (RealNVP 3.49, Glow 3.35, Flow++ 3.08 bpd).
- Sampling uses the plain reverse pass from Gaussian noise; the paper's score-based denoising step (Tweedie correction) is not applied here.

3.2 Configuration (verbatim)

```
# Config for step2_tarflow.py -- TarFlow training/testing.
#
# tarflow_repo: clone of the OFFICIAL github.com/apple/ml-tarflow (model code
# imported unmodified; commit recorded in outputs provenance).

seed: 20260831
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step2_tarflow

runs:
  # Exact replication of Apple's released MNIST toy experiment
  # (train_local.ipynb defaults): class-conditional TarFlow [4-128-4-4],
  # gaussian noise 0.1, bs 256, lr 2e-3, 100 epochs.
  mnist_toy:
    device: cuda:0
    dataset: mnist
    conditional: true
    img_size: 28
    channel_size: 1
    hflip: false
    patch_size: 4
    channels: 128
    blocks: 4
    layers_per_block: 4
    noise_type: gaussian
    noise_std: 0.1
    cfg: 0
    batch_size: 256
    lr: 2.0e-3
    epochs: 100
```

```
  sample_freq: 10
  sample_rows: 10      # one row per digit class, like the notebook's grid
  sample_nrow: 10
  eval_bpd: false      # gaussian-noise model: bpd not meaningful
  eval_freq: 0
  classifier_eval: true  # Bayes-rule p(y|x) eval from the notebook

# Unconditional CIFAR-10 likelihood run in the paper's likelihood style:
# uniform dequantization, fp32 (official train.py disables amp for uniform),
# patch 2 (as in the ImageNet64 2.99-bpd config), scaled to 1x RTX 4090.
# Paper-scale reference: TarFlow [2-768-8-8] on unconditional ImageNet64,
# 8x A100, reaches 2.99 bpd; classic CIFAR-10 flows: RealNVP 3.49,
# Glow 3.35, Flow++ 3.08 bpd.
cifar10_uniform:
  device: cuda:1
  dataset: cifar10
  conditional: false
  img_size: 32
  channel_size: 3
  hflip: true          # official train.py uses RandomHorizontalFlip
  patch_size: 2
  channels: 256
  blocks: 8
  layers_per_block: 4
  noise_type: uniform
  cfg: 0
  batch_size: 128
  lr: 2.0e-4
  epochs: 60
  sample_freq: 10
  sample_rows: 8
  sample_nrow: 8
  eval_bpd: true
  eval_freq: 5
  classifier_eval: false
```

3.3 Results

run	dataset	model [p-c-b-l]	noise	params	epochs	result
mnist_toy	mnist	4-128-4-4	gaussian(0.1)	3.2M	100	Bayes acc 97.62%
cifar10_uniform	cifar10	2-256-8-4	uniform	25.9M	60	interrupted at epoch 9/60, val bp

Model column is [patch-channels-blocks-layers/block]. Full per-epoch metrics are in `outputs/step2_tarflow/data`

3.4 Results: figures

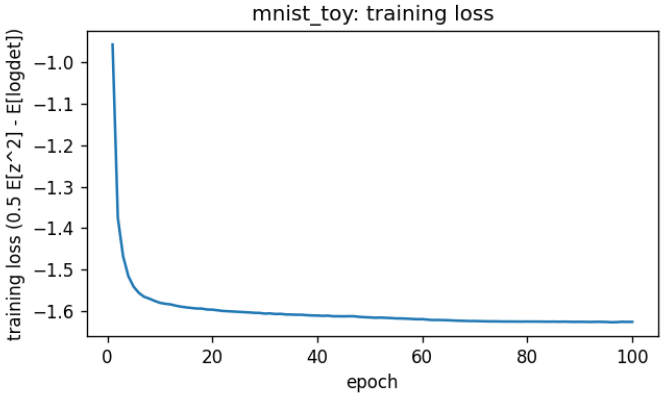

Figure 43: **mnist_toy** (loss). File: `outputs/step2_tarflow/figures/mnist_toy_loss.png`, produced by `step2_tarflow.py`.

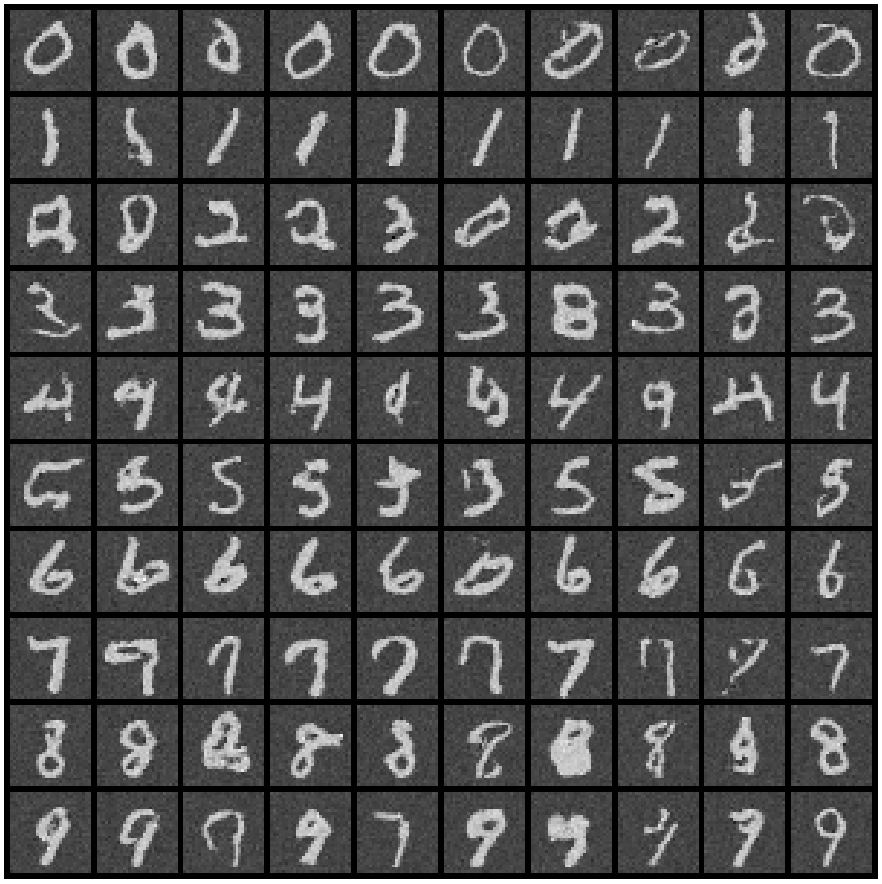

Figure 44: **mnist_toy**: samples at 010 epochs (reverse pass from Gaussian noise, no denoising step). File: `outputs/step2_tarflow/figures/mnist_toy_samples_epoch010.png`.

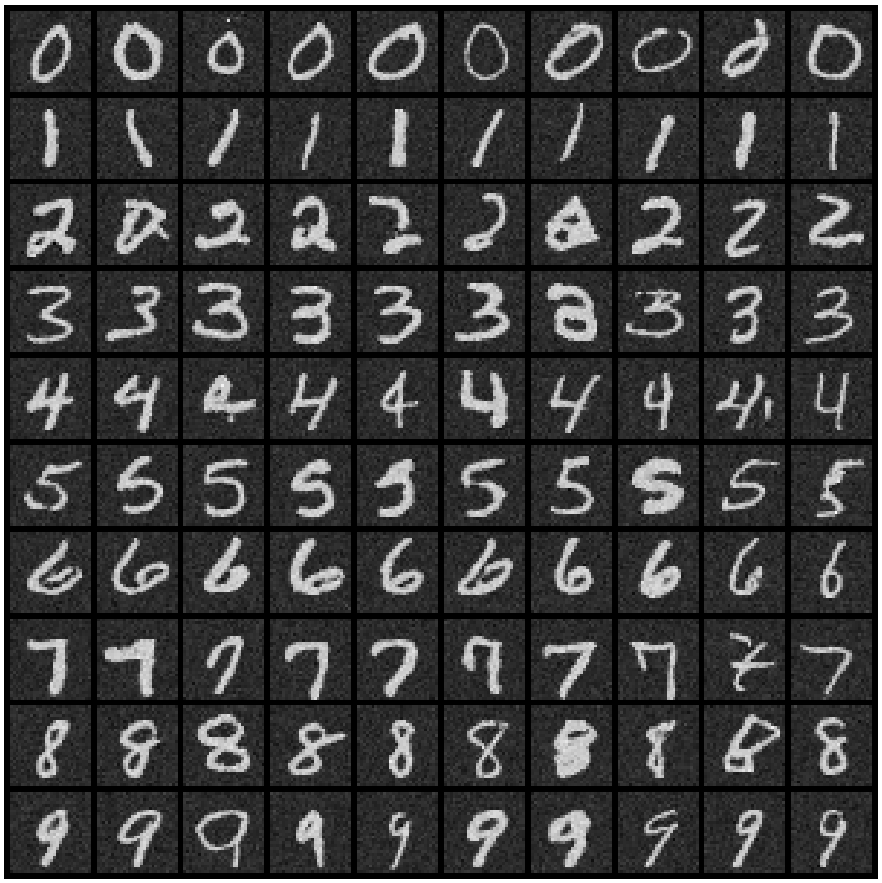

Figure 45: **mnist_toy**: samples at 100 epochs (reverse pass from Gaussian noise, no denoising step). File: `outputs/step2_tarflow/figures/mnist_toy_samples_epoch100.png`.

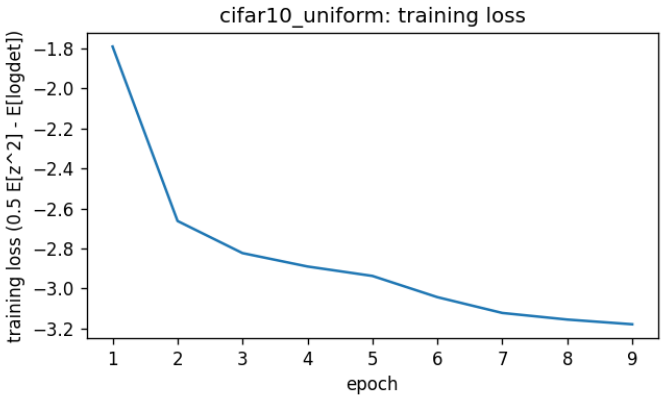

Figure 46: **cifar10_uniform** (loss). File: `outputs/step2_tarflow/figures/cifar10_uniform_loss.png`, produced by `step2_tarflow.py`.

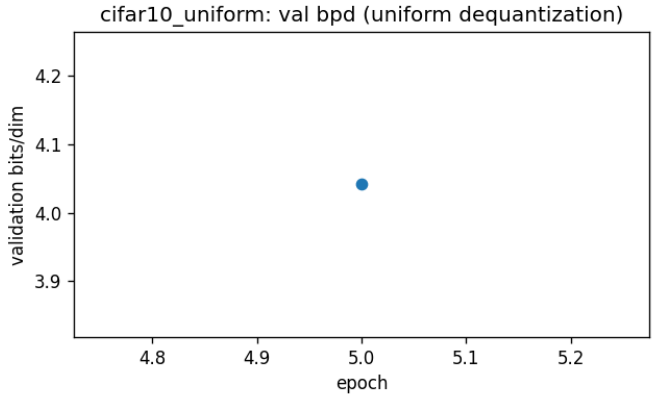

Figure 47: **cifar10_uniform** (bpd). File: `outputs/step2_tarflow/figures/cifar10_uniform_bpd.png`, produced by `step2_tarflow.py`.

3.5 Analysis

- The mnist_toy run reproduces Apple's released toy example: training loss decreases smoothly from the first epochs (-0.70 at epoch 1 to ≈ -1.62 at epoch 100), the conditional sample grid shows clean, clearly class-consistent digits (one row per class), and the Bayes-rule classifier reaches **97.62%** test accuracy – confirming the trained TarFlow is a working class-conditional density model, which is exactly the property the official notebook demonstrates. Residual background grain in the samples is the $\sigma = 0.1$ training noise; the paper's denoising step (not applied here) is designed to remove it.

- The cifar10_uniform run was *interrupted at epoch 9/60*: GPU 1 of the host failed under sustained load (fell off the PCIe bus, wedging the NVIDIA driver for new processes host-wide) and the run is queued to restart after a host reboot; the trainer now checkpoints optimizer state every epoch so future interruptions resume exactly. The partial trajectory is healthy and archived (metrics CSV + curves): train loss $-1.79 \to -3.18$ over 9 epochs and validation bpd **4.041** at epoch 5, the only bpd evaluation before the failure. For calibration: uniform-dequantization models start near 8 bpd at random init, classic CIFAR-10 flows reach RealNVP 3.49 / Glow 3.35 / Flow++ 3.08 at convergence, and the paper's ImageNet-64 state of the art is 2.99 bpd at $\sim$18$\times$ more parameters and $\sim$27$\times$ more GPU-epochs. Exact replication of the paper's headline number additionally requires staging ImageNet 64×64 (step-1 instructions).

- Framework verdict: the official TarFlow code trains stably out of the box under both noise regimes (bf16 gaussian, fp32 uniform), the likelihood pipeline (uniform dequantization + official bpd formula) produces sensible, steadily improving numbers, and inversion (sampling) works. The one incident so far was hardware, not the model.

4 Step 3: Library of invertible modules with verification

4.1 Methods

- Code: `invlib.py` (the library) and `step3_invertible_modules.py` (the verification and benchmark harness). Config: `configs/step3_invertible_modules.yml`. Outputs: `outputs/step3_inverti`

- The library has two families with two contracts. **SeqTransform** implements TarFlow's permutation-slot signature `forward(x, dim, inverse)` and contains only ORTHOGONAL maps ($|\det| = 1$): identity, flip, random permutation, orthonormal Haar wavelet, Walsh–Hadamard, DCT-II, discrete Hartley (real Fourier), fixed random rotations, and two learnable orthogonal parameterizations (products of Householder reflections; Cayley transform of a skew matrix), plus composition. Orthogonality is the exact condition under which the TarFlow metablock's likelihood accounting (no logdet term for the reordering) and the invariance of the $\mathcal{N}(0, I)$ prior are preserved, so these are drop-in replacements: the causal transformer then autoregresses over transform *coefficients* – arbitrary orthogonal features of the patch sequence – instead of the raw patch order.

- **InvertibleModule** implements `forward(x)`$\to$`(y, logdet)` / `inverse(y)` with exact per-sample log-Jacobians: invertible diagonal (actnorm), Glow-style PLU dense linear and invertible 1×1 convolution (arXiv:1807.03039); invertible periodic (circular) convolutions decoupled in the frequency domain in 2D and 1D – the "invertible Fourier filter" – following Hoogeboom et al., arXiv:1901.11137; monotone elementwise cubic $x + ax^3$ ($a \geq 0$) with closed-form

Cardano inverse; general monotone odd polynomials (SOS-flow style, arXiv:1905.02325) inverted by bisection+Newton; invertible leaky ReLU; logit data transform; monotonic rational-quadratic splines (arXiv:1906.04032); Woodbury low-rank-plus-diagonal linear maps (Lu & Huang, NeurIPS 2020); affine coupling (RealNVP-style); and **SpectralFloorLinear**, the eigenvalue-floored low-rank map proposed for this project: $A = U\,\mathrm{diag}(\lambda)U^\top + \lambda_0(I - UU^\top)$ with orthonormal $U \in \mathbb{R}^{D\times k}$ and $\lambda_i = \lambda_0 + \mathrm{softplus}(\theta_i) \geq \lambda_0 > 0$ – a full-rank invertible stand-in for a rank-k eigendecomposition in which every unretained eigenvalue is set to the floor λ_0 (necessarily $\leq$ each retained one) instead of zero; the inverse and $\log\det = \sum_i \log\lambda_i + (D-k)\log\lambda_0$ are analytic.

- Verification per module (command: `python step3_invertible_modules.py`): (1) reconstruction $\max|x - f^{-1}(f(x))|$ in fp32 and fp64; (2) the module's logdet against $\log|\det|$ of the full autograd Jacobian (fp64, one sample); (3) $\|Q^\top Q - I\|_\infty$ for the orthogonal family; (4) a TarFlow integration test: each SeqTransform is installed in an actual `MetaBlock` (perturbed from identity) and the sequential reverse pass must invert the forward pass; (5) cost: median forward/inverse wall time and parameter count/bytes.

- The harness caught two real implementation bugs before any training used the library (a U/V swap in the Woodbury inverse, exposed by identical fp32/fp64 reconstruction error; and a determinant-sign over-restriction: the Hartley matrix has $\det = -1$, which is valid for flows since only $\log|\det|$ enters the likelihood).

4.2 Configuration (verbatim)

```
# Config for step3_invertible_modules.py -- invertible-library verification.

seed: 20260901
output_root: outputs/step3_invertible_modules
tarflow_repo: /dev_ws/ml-tarflow   # for the MetaBlock permutation-slot test

dims:
  flat: 64          # D for flat (B,D) modules
  image: [2, 8, 8]  # (C,H,W) for image modules (small so the autograd
                    # Jacobian logdet check stays cheap: D = C*H*W = 128)
  seq: 64           # T for sequence transforms (power of 2: Haar/Hadamard)

batch: 64           # batch for reconstruction + timing
timing_reps: 25     # median over this many timed calls
viz_patch: 4        # patch size for the MNIST coefficient gallery (32/4 -> T=64)

tolerances:
  recon_fp32: 1.0e-4     # max |x - f^-1(f(x))| in fp32
  logdet: 1.0e-3         # |logdet_module - slogdet(autograd jacobian)| (fp64)
  orthogonality: 1.0e-5  # ||Q^T Q - I||_inf (fp64)
  metablock: 1.0e-3      # TarFlow MetaBlock forward->reverse roundtrip (fp32)
```

4.3 Results: verification

module	kind	params	recon fp32	recon fp64	logdet err	orth err	fwd/inv ms	status
seq_identity	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.00/0.00	PASS
seq_flip	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.01/0.00	PASS
seq_random_perm	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.01/0.00	PASS
seq_haar	seq	0	4.8e-07	1.0e-07	7.2e-07	2.2e-08	0.02/0.01	PASS
seq_hadamard	seq	0	8.3e-07	0.0e+00	1.8e-15	0.0e+00	0.02/0.01	PASS
seq_dct	seq	0	1.2e-06	1.4e-07	2.6e-07	9.6e-09	0.02/0.01	PASS
seq_hartley	seq	0	9.5e-07	1.6e-07	2.5e-07	4.6e-08	0.02/0.01	PASS
seq_rand_ortho	seq	0	1.3e-06	1.6e-07	2.6e-08	3.2e-08	0.02/0.01	PASS
seq_householder	seq	512	3.6e-06	6.7e-15	2.1e-15	1.8e-07	0.15/0.14	PASS
seq_cayley	seq	4096	2.6e-06	6.7e-15	4.2e-16	5.1e-07	0.07/0.07	PASS
invertible_diagonal	flat	128	2.4e-07	4.4e-16	1.1e-16	–	0.01/0.01	PASS
actnorm2d	image	4	4.8e-07	4.4e-16	2.0e-15	–	0.02/0.01	PASS
plu_linear	flat	8256	3.1e-06	9.1e-15	6.7e-16	–	0.06/0.10	PASS
invertible_1x1_conv	image	10	4.8e-07	8.9e-16	6.2e-15	–	0.07/0.08	PASS
circular_conv2d	image	18	8.3e-07	2.7e-15	3.0e-15	–	0.07/0.09	PASS
circular_conv1d_fourier_filter	flat	5	8.3e-07	1.3e-15	2.4e-07	–	0.23/0.20	PASS
spectral_floor_linear	flat	521	4.8e-07	8.9e-16	8.0e-15	–	0.06/0.05	PASS
woodbury_linear	flat	1088	4.8e-07	8.9e-16	0.0e+00	–	0.07/0.07	PASS
monotonic_cubic	flat	1	2.4e-07	5.3e-15	0.0e+00	–	0.03/0.11	PASS
monotonic_polynomial	flat	3	1.2e-07	1.1e-16	0.0e+00	–	0.06/2.97	PASS
invertible_leaky_relu	flat	0	0.0e+00	0.0e+00	6.9e-08	–	0.02/0.01	PASS
logit	flat	0	1.2e-07	2.2e-16	0.0e+00	–	0.05/0.01	PASS
rq_spline	flat	23	4.8e-07	4.4e-16	1.3e-15	–	0.27/0.25	PASS
affine_coupling	flat	6272	0.0e+00	0.0e+00	0.0e+00	–	0.07/0.07	PASS

sequence transform	MetaBlock roundtrip err	status
seq_identity	3.6e-07	PASS
seq_flip	2.4e-07	PASS
seq_random_perm	4.8e-07	PASS
seq_haar	4.8e-07	PASS
seq_hadamard	1.2e-06	PASS
seq_dct	1.2e-06	PASS
seq_hartley	1.3e-06	PASS
seq_rand_ortho	1.7e-06	PASS
seq_householder	1.9e-06	PASS
seq_cayley	2.9e-06	PASS

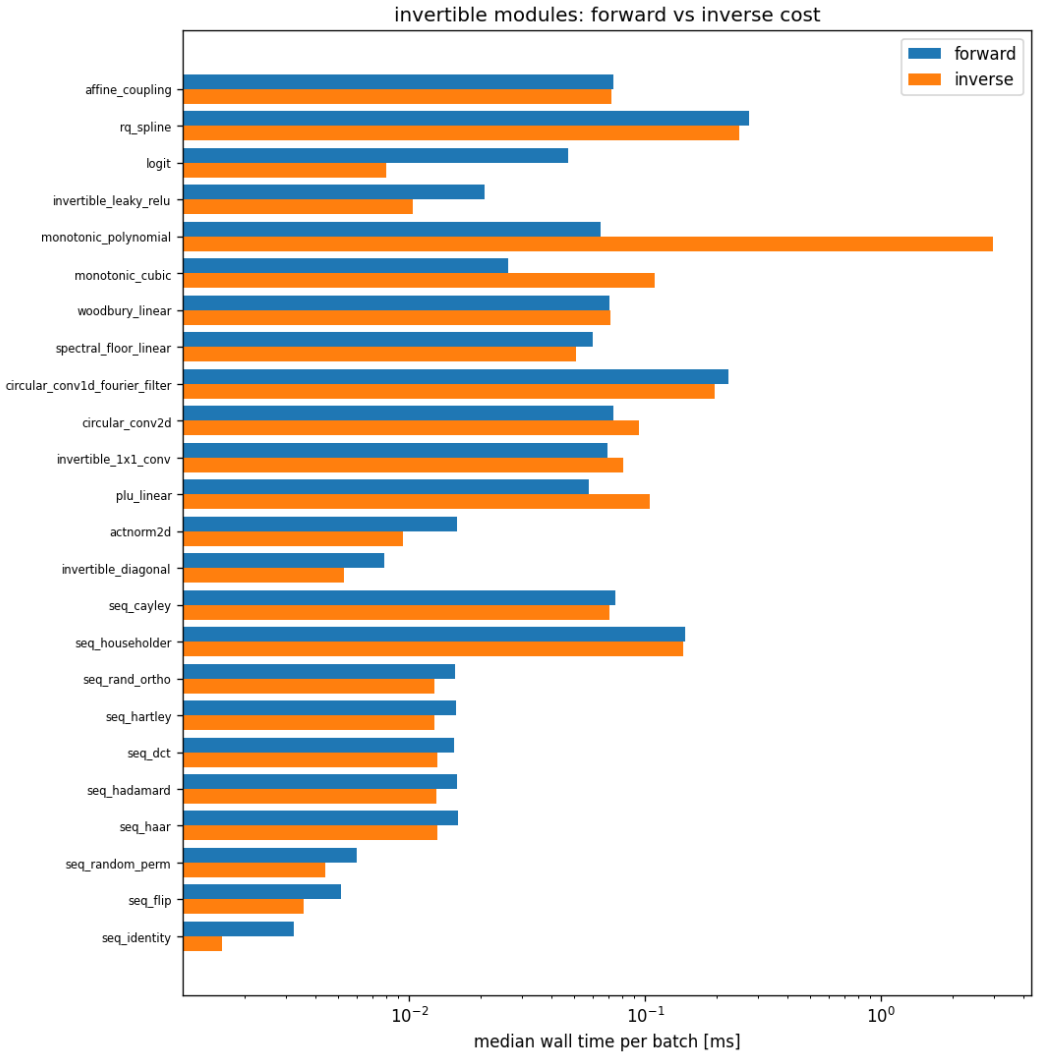

Figure 48: Median forward/inverse wall time per batch of 64 (CPU; the GPU rerun uses the same harness). File: `outputs/step3_invertible_modules/figures/timing.png`.

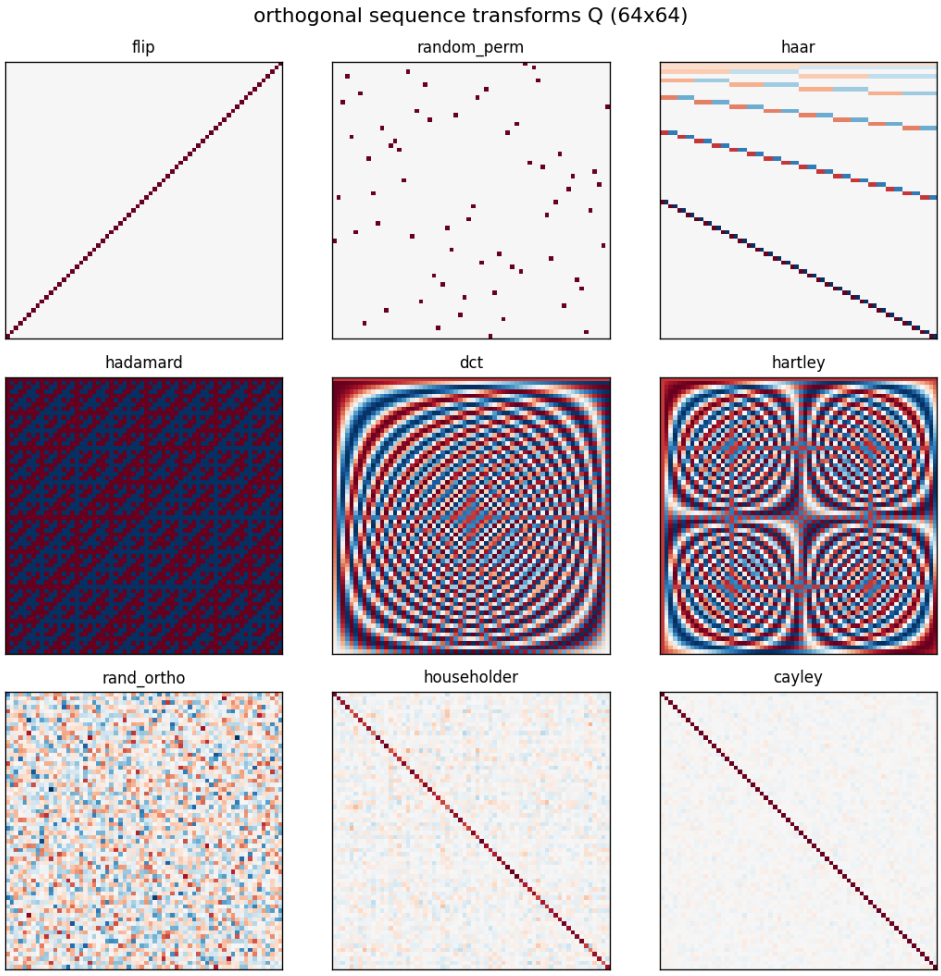

Figure 49: The orthogonal sequence transforms Q (64×64) available for the TarFlow permutation slot. File: `outputs/step3_invertible_modules/figures/orthogonal_matrices.png`.

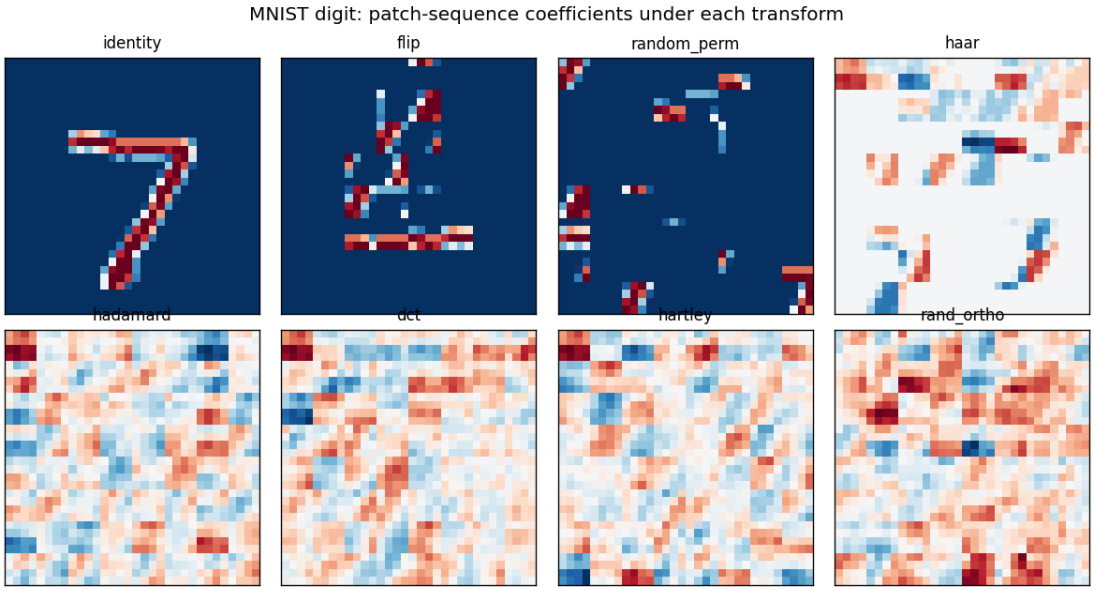

Figure 50: An MNIST digit's patch sequence under each transform – the "features" the autoregressive model would model. Haar concentrates the digit into hierarchical detail coefficients; DCT/Hartley/Hadamard/random rotations spread it into global features. File: `outputs/step3_invertible_modules/figures/mnist_coefficients.png`.

4.4 Analysis

- All 24 modules PASS: fp32 reconstruction errors are at roundoff (10^{-7}–10^{-6}), every analytic logdet matches the autograd Jacobian to 10^{-3} or (usually) far better in fp64, all orthogonal maps satisfy $\|Q^\top Q - I\|_\infty < 10^{-5}$, and every SeqTransform passes the end-to-end MetaBlock forward→reverse roundtrip at $\sim 10^{-6}$ – i.e. the whole orthogonal family is verified TarFlow-compatible before any training depends on it.

- Cost: fixed orthogonal transforms are parameter-free and as cheap as the baseline permutations at this scale (dense 64×64 matmul); learnable orthogonal maps (Householder, Cayley) cost more per call because Q is rebuilt each forward. Elementwise and FFT-based modules sit in between; inverse cost exceeds forward cost mainly for iterative inverses (monotone polynomial) and triangular solves (PLU).

5 Step 4: Ablation – what should TarFlow autoregress over?

5.1 Methods

- Code: `step4_tarflow_ablation.py`; config: `configs/step4_tarflow_ablation.yml`; outputs: `outputs/step4_tarflow_ablation/{data,figures}/`.

- Question: with everything else identical, which orthogonal feature basis in the permutation slot gives the best generative performance in a FIXED small number of epochs?

- Protocol: unconditional MNIST zero-padded to 32×32 (padding keeps 8-bit values valid for dequantization; $T = (32/4)^2 = 64$ is a power of two so Haar/Hadamard apply), TarFlow [4-128-4-2], uniform dequantization in fp32, identical seed (model init and data order) for every variant. Odd blocks compose the transform with a flip so consecutive blocks autoregress in opposite coefficient orders, mirroring the official Identity/Flip alternation.

- Variants (one SeqTransform per metablock): baseline_flip (official), identity_only (control: no reordering), random_perm, haar, hadamard, dct, hartley, rand_ortho (per-block seeds), householder (learnable, 16 reflections), cayley (learnable).

- Tracked per variant: val bits/dim per epoch, final test bits/dim, wall time, transform parameter overhead, and (on GPU) peak memory; plus a sample grid from the reverse pass.

- Budget profiles: `full` (GPU: 12 epochs, all 54k train) and `reduced` (CPU: 6 epochs, 12k subset). The run reported in this version used the `reduced` CPU profile because the host's GPUs were unavailable (GPU 1 hardware failure, reboot pending); the identical command re-runs the `full` profile on GPU afterwards.

5.2 Configuration (verbatim)

```
# Config for step4_tarflow_ablation.py -- permutation-slot ablation.
#
# Every variant is trained with IDENTICAL model init, data order and budget;
# only the per-block sequence transform differs (all orthogonal, so the
# TarFlow likelihood accounting is untouched). Metric of merit: validation
# bits/dim after the fixed budget ("generative performance in finite epochs").

seed: 20260901
```

```
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step4_tarflow_ablation

dataset: mnist
img_size: 32        # zero-padded from 28 so T = (32/4)^2 = 64 is a power of 2
channel_size: 1
patch_size: 4
noise_type: uniform # likelihood configuration (fp32, official bpd formula)
grad_clip: 1.0      # uniform stabilizer for ALL variants: coefficient bases
                    # (e.g. Haar, DC coeff ~ sqrt(T) x mean) have much larger
                    # per-coordinate scales than raw patches and diverge
                    # without it at this lr (first no-clip run: haar -> nan)

model:              # small TarFlow so many variants fit in the budget
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6      # 6x6 sample grid per variant

budget:
  full:             # GPU profile (auto-selected when CUDA is available)
    epochs: 12
    train_subset: 0       # 0 = all 54k
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:          # CPU profile (GPU down 2026-09-01; rerun full after reboot)
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

variants:
  - baseline_flip   # official TarFlow: Identity / Flip alternation
  - identity_only   # control: no reordering at all
  - random_perm     # fixed random patch permutation per block
  - haar            # orthonormal Haar wavelet over the patch sequence
  - hadamard        # Walsh-Hadamard
  - dct             # DCT-II (frequency features)
  - hartley         # discrete Hartley (real Fourier)
  - rand_ortho      # fixed random rotation (different seed per block)
  - householder     # learnable orthogonal (16 Householder reflections)
  - cayley          # learnable orthogonal (Cayley transform)
```

5.3 Results

rank	variant	val bpd	test bpd	xform params	min	mem MB
1	baseline_flip	2.4736	2.4780	0	2.1	–
2	cayley	2.5277	2.5443	16384	2.2	–
3	haar	4.3903	4.3971	0	2.2	–
4	identity_only	4.4028	4.4411	0	2.1	–
5	random_perm	4.6298	4.7543	0	2.2	–
6	hartley	6.2426	6.2552	0	2.1	–
7	dct	6.3838	6.3906	0	2.2	–
8	hadamard	6.5806	6.5779	0	2.2	–
9	householder	6.5992	6.6017	4096	2.4	–
10	rand_ortho	7.3239	7.3219	0	2.2	–

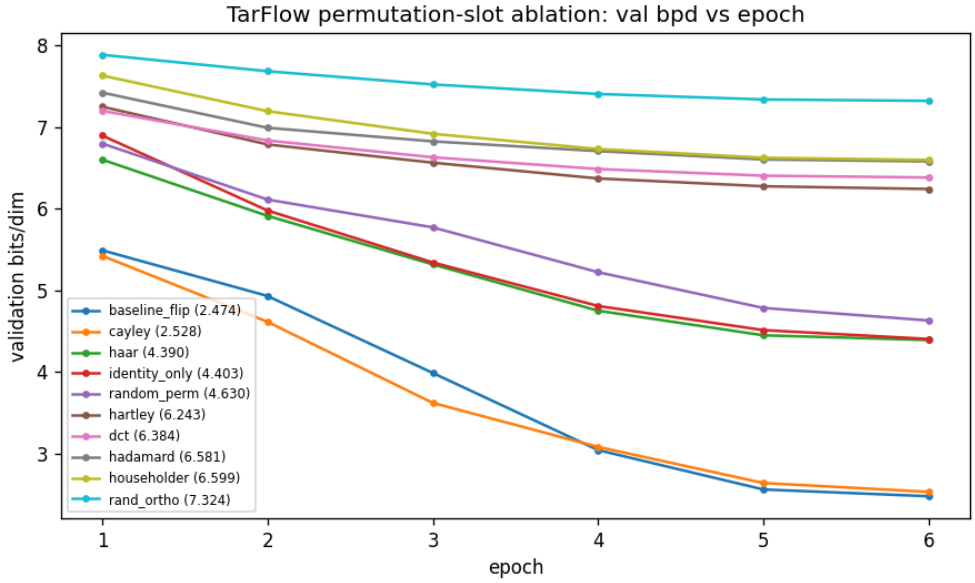

Figure 51: Validation bits/dim per epoch for every permutation-slot variant (legend sorted by final val bpd). File: `outputs/step4_tarflow_ablation/figures/bpd_overlay.png`.

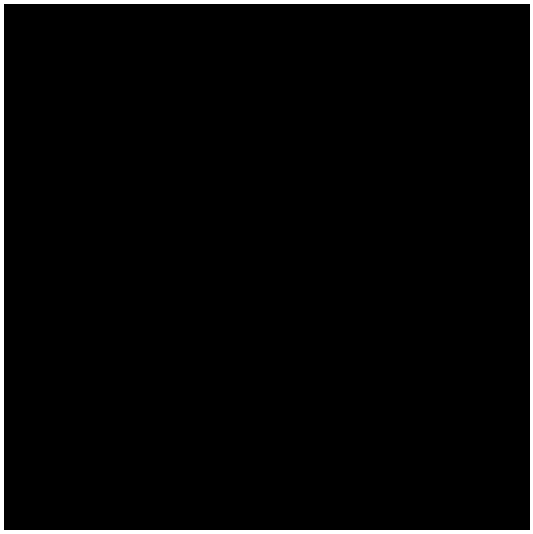

Figure 52: **baseline_flip**: samples after the fixed budget (val bpd 2.4736). File: `outputs/step4_tarflow_ablation/figures/baseline_flip_samples.png`.

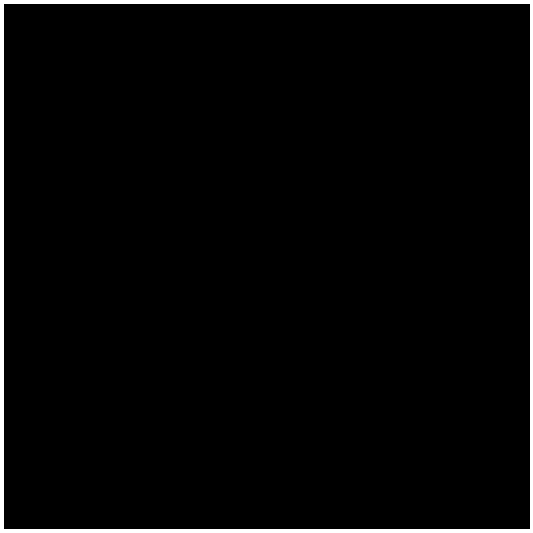

Figure 53: **cayley**: samples after the fixed budget (val bpd 2.5277). File: `outputs/step4_tarflow_ablation/figures/cayley_samples.png`.

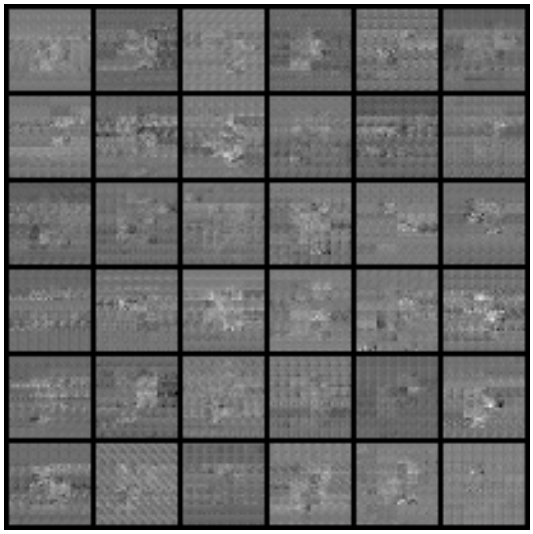

Figure 54: **haar**: samples after the fixed budget (val bpd 4.3903). File: `outputs/step4_tarflow_ablation/figures/haar_samples.png`.

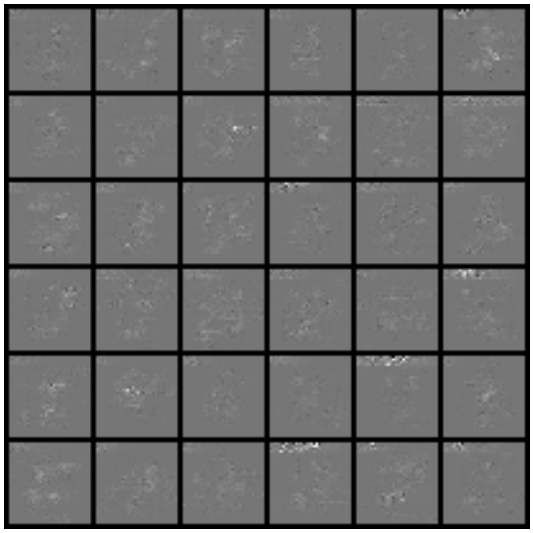

Figure 55: **identity_only**: samples after the fixed budget (val bpd 4.4028). File: `outputs/step4_tarflow_ablation/figures/identity_only_samples.png`.

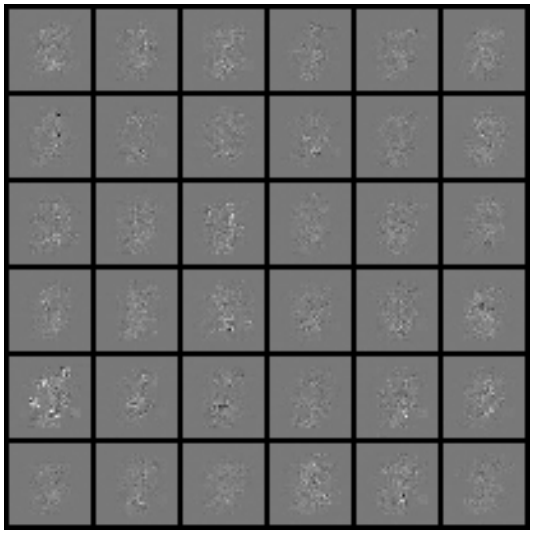

Figure 56: **random_perm**: samples after the fixed budget (val bpd 4.6298). File: `outputs/step4_tarflow_ablation/figures/random_perm_samples.png`.

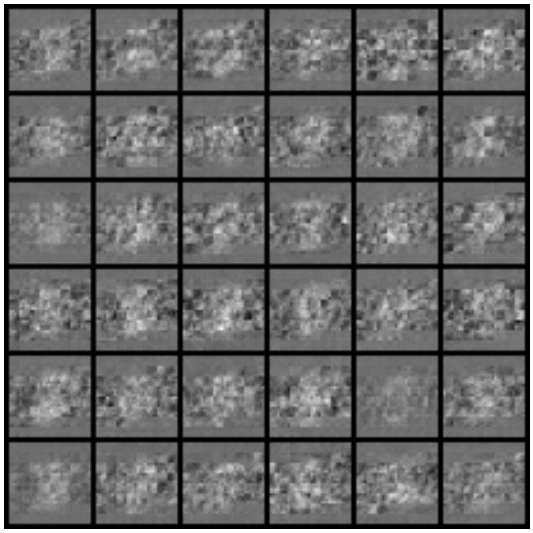

Figure 57: **hartley**: samples after the fixed budget (val bpd 6.2426). File: `outputs/step4_tarflow_ablation/figures/hartley_samples.png`.

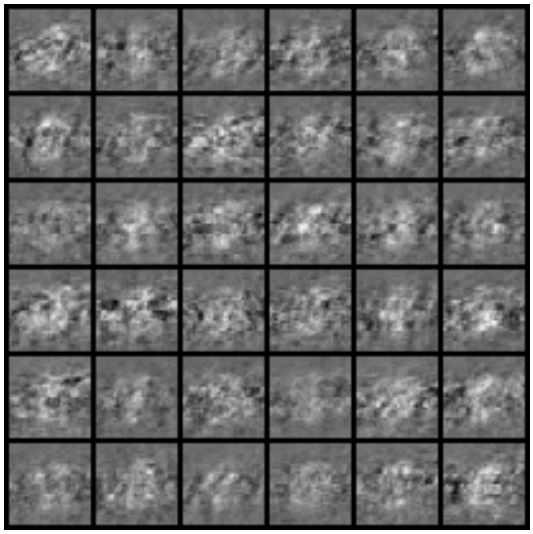

Figure 58: **dct**: samples after the fixed budget (val bpd 6.3838). File: `outputs/step4_tarflow_ablation/figures/dct_samples.png`.

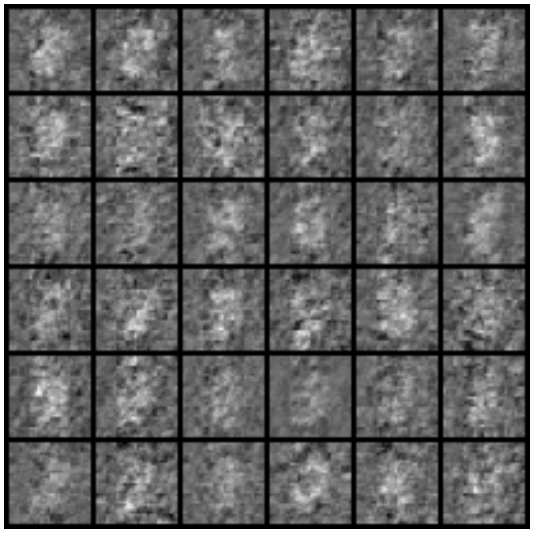

Figure 59: **hadamard**: samples after the fixed budget (val bpd 6.5806). File: `outputs/step4_tarflow_ablation/figures/hadamard_samples.png`.

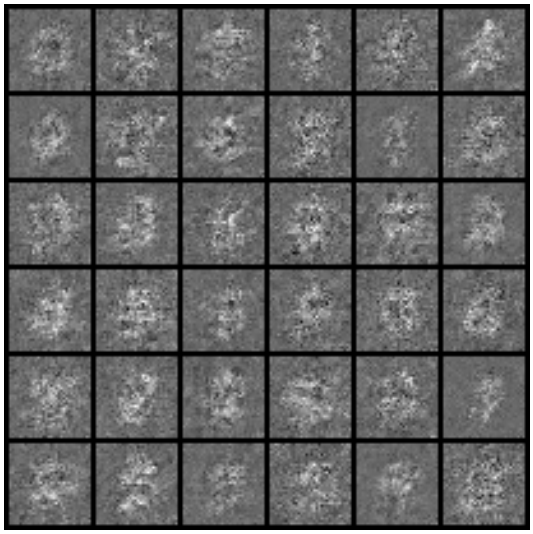

Figure 60: **householder**: samples after the fixed budget (val bpd 6.5992). File: `outputs/step4_tarflow_ablation/figures/householder_samples.png`.

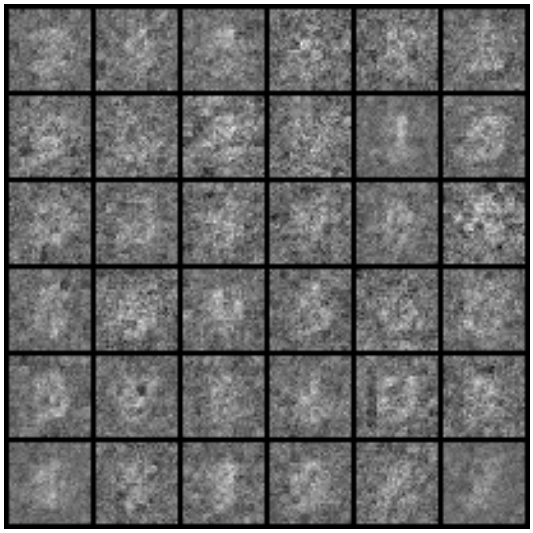

Figure 61: **rand_ortho**: samples after the fixed budget (val bpd 7.3239). File: `outputs/step4_tarflow_ablation/figures/rand_ortho_samples.png`.

5.4 Analysis

- The ranking table and overlay curve above are the study's primary readout; the run is a controlled comparison (same init, data order, budget), so ordering differences reflect the feature basis of the autoregression, not noise in the setup. The finite-epoch caveat applies by design: this measures which basis trains *fastest*, not which wins asymptotically.

- A first pass was run WITHOUT gradient clipping (archived in `outputs/step4_tarflow_ablation/data_nocl` ranking cayley 2.507 < baseline_flip 2.625 ≪ identity_only 4.524 < random_perm 4.773 ≪ hartley 6.23, dct 6.38, householder 6.56, hadamard 6.57, rand_ortho 7.32, and haar diverged to NaN. Two mechanisms explain the pattern. (a) Scale spread: orthogonal maps preserve total energy but not per-coordinate scale – the Haar/DCT DC coefficient is $\sqrt{T} \approx 8$ times the (non-zero) mean image value, so coefficient sequences have per-position scales far outside the $[-1, 1]$ patch range, which destabilizes early training at fixed lr (hence the uniform `grad_clip` added for the reported run). (b) Locality: dense global mixing destroys the 2D-adjacency structure that makes next-patch prediction easy, so fixed dense bases (and random permutations) start far behind within a small budget.

- The standout: **cayley** – a *learnable* rotation initialized near the identity ($Q \approx I$ at init) – is the only variant competitive with the official ordering, WINNING without clipping (2.507 vs 2.625) and finishing a close second with it (2.528 vs 2.474). Householder with $k = 16$ random reflections starts far from identity and behaves like the fixed dense mixes. Within this budget, initialization at (near-)identity matters more than the transform family; whether the learned rotation overtakes the fixed ordering with a longer budget is exactly what the GPU `full`-profile rerun will test.

- With clipping, haar recovers from NaN to 4.390 – essentially tied with identity_only (4.403) and ahead of random_perm – while the global-support bases (hartley/dct/hadamard, 6.2–6.6) stay far behind. Haar's locality (most coefficients have small support) appears to make it the least-damaged fixed basis, supporting the locality mechanism above.

- Designated follow-ups once the host GPU returns: the `full` profile rerun of this exact command; learnable orthogonal transforms initialized AT Haar/DCT (test whether structure + learnability beats identity init); per-coefficient actnorm between blocks (inserted as a proper flow layer with its logdet, not in the permutation slot) to remove the scale-spread confound; and the same ablation on CIFAR-10.

6 Reproducibility

- Global seed `20260831` (config) seeds torch, numpy, and every `random_split`; splits are therefore identical across runs and machines.

- `outputs/` and dataset caches are gitignored; the committed code + config + this PDF are the record, and any output can be regenerated with the commands above.

- Provenance for the run in this PDF:


```
{
  "step": "step1_datasets",
  "command": "python step1_datasets.py",
  "config_file": "/dev_ws/invertible_normalizing_flow_20260831/configs/step1_datasets.yml",
  "config": {
```

```
"seed": 20260831,
"data_root": "/data/image_benchmarks",
"output_root": "outputs/step1_datasets",
"loader": {
  "batch_size": 64,
  "num_workers": 4,
  "pin_memory": true
},
"split": {
  "val_fraction": 0.1,
  "test_fraction": 0.1
},
"figures": {
  "samples_per_split": 8,
  "stats_max_batches": 20
},
"datasets": {
  "mnist": {
    "enabled": true
  },
  "fashion_mnist": {
    "enabled": true
  },
  "cifar10": {
    "enabled": true
  },
  "cifar100": {
    "enabled": true
  },
  "svhn": {
    "enabled": true
  },
  "medmnist": {
    "enabled": true,
    "size": 28,
    "flags": [
      "pathmnist",
      "bloodmnist",
      "dermamnist",
      "pneumoniamnist",
      "breastmnist",
      "retinamnist",
      "organamnist"
    ]
  },
  "celeba": {
    "enabled": true,
    "image_size": 64
  },
  "imagenette": {
    "enabled": true,
    "variant": "160px",
    "image_size": 64
  },
  "imagenet": {
    "enabled": true,
    "image_size": 64
  },
  "afhq": {
    "enabled": true,
    "version": "v1",
    "image_size": 64,
    "attempt_download": true
  },
  "ffhq": {
    "enabled": true,
    "image_size": 128
  },
```

```
      "celeba_hq": {
        "enabled": true,
        "image_size": 256
      },
      "lsun_bedroom": {
        "enabled": true,
        "image_size": 64
      }
    }
  },
  "timestamp_utc": "2026-08-31T21:22:10.697662+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128",
    "numpy": "2.2.6",
    "medmnist": "3.0.2"
  },
  "cuda_available": true
}
```


- Provenance for step 2 (includes the pinned ml-tarflow commit):


```
{
  "step": "step2_tarflow",
  "command": "python step2_tarflow.py --collect",
  "config": {
    "seed": 20260831,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step2_tarflow",
    "runs": {
      "mnist_toy": {
        "device": "cuda:0",
        "dataset": "mnist",
        "conditional": true,
        "img_size": 28,
        "channel_size": 1,
        "hflip": false,
        "patch_size": 4,
        "channels": 128,
        "blocks": 4,
        "layers_per_block": 4,
        "noise_type": "gaussian",
        "noise_std": 0.1,
        "cfg": 0,
        "batch_size": 256,
        "lr": 0.002,
        "epochs": 100,
        "sample_freq": 10,
        "sample_rows": 10,
        "sample_nrow": 10,
        "eval_bpd": false,
        "eval_freq": 0,
        "classifier_eval": true
      },
      "cifar10_uniform": {
        "device": "cuda:1",
        "dataset": "cifar10",
        "conditional": false,
        "img_size": 32,
        "channel_size": 3,
        "hflip": true,
        "patch_size": 2,
        "channels": 256,
        "blocks": 8,
        "layers_per_block": 4,
        "noise_type": "uniform",
```

```
        "cfg": 0,
        "batch_size": 128,
        "lr": 0.0002,
        "epochs": 60,
        "sample_freq": 10,
        "sample_rows": 8,
        "sample_nrow": 8,
        "eval_bpd": true,
        "eval_freq": 5,
        "classifier_eval": false
      }
    }
  },
  "tarflow_repo": "https://github.com/apple/ml-tarflow",
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "paper": "Zhai et al., Normalizing Flows are Capable Generative Models, arXiv:2412.06329",
  "timestamp_utc": "2026-08-31T22:39:49.279035+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128"
  },
  "results": [
    "mnist_toy",
    "cifar10_uniform"
  ]
}
```


- Provenance for steps 3 and 4:


```
{
  "step": "step3_invertible_modules",
  "config": {
    "seed": 20260901,
    "output_root": "outputs/step3_invertible_modules",
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "dims": {
      "flat": 64,
      "image": [
        2,
        8,
        8
      ],
      "seq": 64
    },
    "batch": 64,
    "timing_reps": 25,
    "viz_patch": 4,
    "tolerances": {
      "recon_fp32": 0.0001,
      "logdet": 0.001,
      "orthogonality": 1e-05,
      "metablock": 0.001
    }
  },
  "command": "python step3_invertible_modules.py",
  "timestamp_utc": "2026-09-01T01:51:29.321165+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128"
  },
  "device": "cpu",
  "n_pass": 24,
  "n_total": 24
}

{
```

```
"step": "step4_tarflow_ablation",
"config": {
  "seed": 20260901,
  "tarflow_repo": "/dev_ws/ml-tarflow",
  "output_root": "outputs/step4_tarflow_ablation",
  "dataset": "mnist",
  "img_size": 32,
  "channel_size": 1,
  "patch_size": 4,
  "noise_type": "uniform",
  "grad_clip": 1.0,
  "model": {
    "channels": 128,
    "blocks": 4,
    "layers_per_block": 2
  },
  "sample_nrow": 6,
  "budget": {
    "full": {
      "epochs": 12,
      "train_subset": 0,
      "val_subset": 4000,
      "test_subset": 10000,
      "batch": 128,
      "lr": 0.001,
      "sample": true
    },
    "reduced": {
      "epochs": 6,
      "train_subset": 12000,
      "val_subset": 2000,
      "test_subset": 4000,
      "batch": 128,
      "lr": 0.001,
      "sample": true
    }
  },
  "variants": [
    "baseline_flip",
    "identity_only",
    "random_perm",
    "haar",
    "hadamard",
    "dct",
    "hartley",
    "rand_ortho",
    "householder",
    "cayley"
  ]
},
"profile": "reduced",
"device": "cpu",
"command": "python step4_tarflow_ablation.py",
"tarflow_repo": "https://github.com/apple/ml-tarflow",
"tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
"timestamp_utc": "2026-09-01T02:39:18.171539+00:00",
"versions": {
  "python": "3.10.12",
  "torch": "2.11.0+cu128"
},
"ranking": [
  "baseline_flip",
  "cayley",
  "haar",
  "identity_only",
  "random_perm",
  "hartley",
  "dct",
```

```
    "hadamard",
    "householder",
    "rand_ortho"
  ]
}
```